ISEN
ALL IS DIGITAL!
OUEST

yncréa

Electronic Practical Work

2026

Microcontoller STM32/Cortex M And Assembly Langage

Jean-Jacques Meneu

Chapter 1. Introduction **3**
1.1. Preparation 3
1.2. Report 3

Chapter 2. Discovering the microcontroller and the IDE **4**
2.1. Microcontroller Overview 4
2.2. Assembly language programming 9
2.3. Cortex M Instructions 15
2.4. Addressing Modes in Cortex M 16
2.5. Development Set 17

Chapter 3. First Steps **19**
3.1. Introduction 19
3.2. Contruction of a modulo 10 counter 29
3.3. How to design a for-loop ? 31
3.4. Create and call subroutines 31
3.5. what is the stack used for? 34
3.6. Using tables 36

Chapter 4. Development Board **38**
4.1. Available Peripherals on the nucleo board 38
4.2. Clock Management 41

Chapter 5. Inputs/Ouputs **44**
5.1. Operating Principle 44
5.2. Initializing input/output ports 45
5.3. Detailed Characteristics 45
5.4. Controlling an LED with a push button 47
5.5. LED flashing 49

Chapter 6. Interrupts **50**
6.1. Principle of handling external interrupts 50
6.2. Flashing with on/off with interrupt 52

Appendix 1. Conditional jump instructions **54**

Appendix 2. Create an STM32 project **55**

Appendix 3. Debugging Windows **59**

Chapitre 1. Introduction

The primary objectives of this series of practical laboratory exercises are as follows:

- **Microcontroller Architecture:** Analyze the core operation and internal architecture of 32-bit microcontrollers using the **STM32L476** from STMicroelectronics. This target features an Arm® Cortex®-M core—the industry-standard architecture for modern embedded systems.

- **Integrated Development Environment (IDE):** Master a complete toolchain for developing embedded C and assembly applications. The environment integrates essential software development tools, including project management, a syntax-highlighting code editor, cross-compilers/assemblers, a linker, a symbolic debugger, and the **NUCLEO-L476RG** hardware development board.

Pedagogical Progression:

You will begin by implementing low-level peripheral drivers and basic applications to build familiarity with the STM32 architecture, register maps, and debugging workflows. Subsequent modules will build upon these foundational exercises to construct more complex, multi-peripheral embedded applications.

Chapter 2. Discovering the microcontroller and the IDE

2.1. Microcontroller Overview

A microcontroller is a microprocessor or CPU around which memories and peripherals have been added.

The figure below shows the STM32L476RG microcontroller.

Figure 1. STM32L476xx block diagram

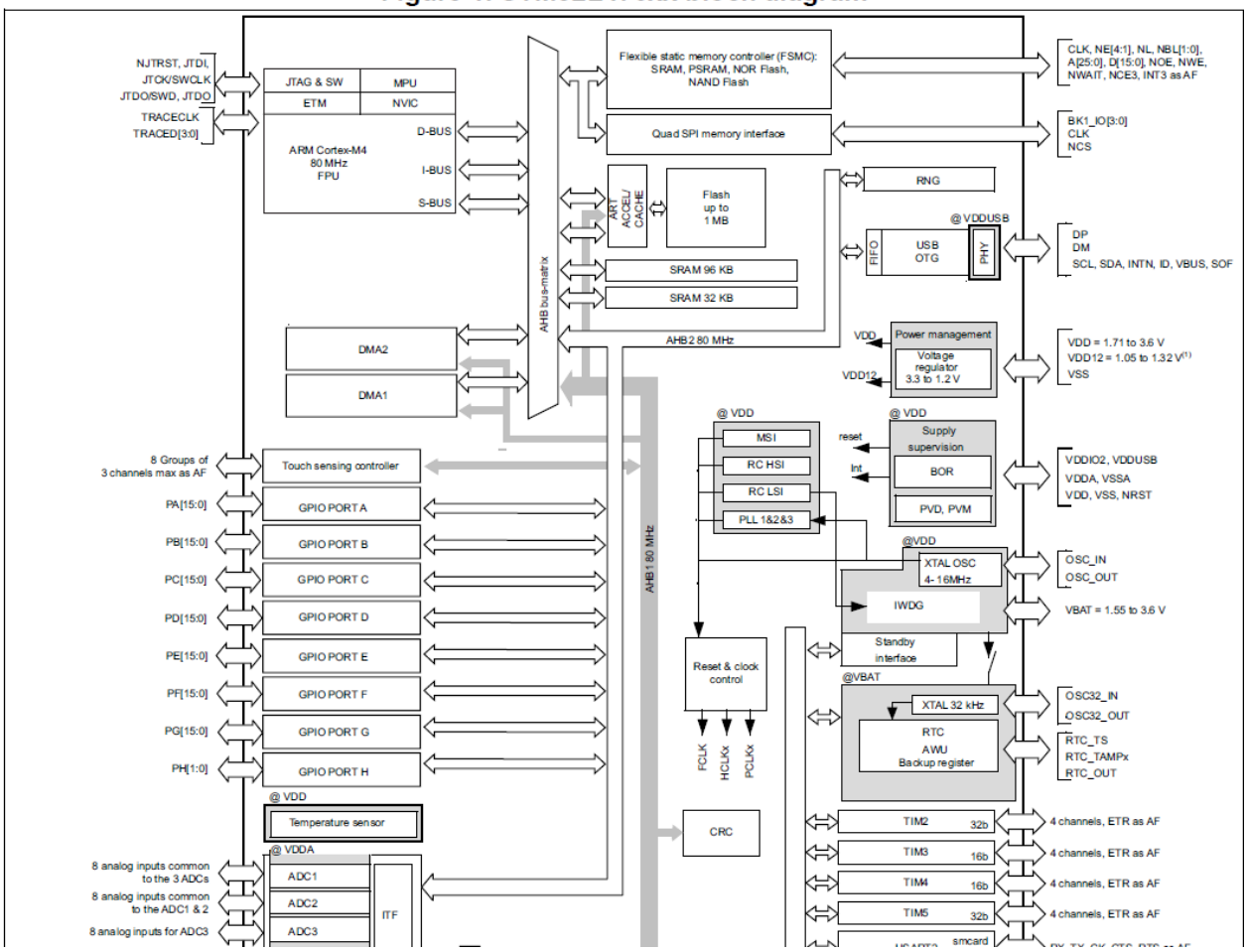


Figure 1. block diagram du STM32L476 (see more details in datasheet)

The CPU features a **Harvard architecture**, utilizing a 32-bit address bus to the program memory and a separate 32-bit address bus between data memory and the core

The **CPU** contains 18 registers, all of 32 bits in size:

- o twelve general registers (R0 to R12)
- o a PC (Program Counter) ordinal counter,
- o a SP (Stack Pointer) stack pointer
- o a LR (Link Register) link register

- o a PSR (Program Status Register) status register
- o a SP (16 bits) stack pointer.

These core registers are common across the entire Arm® Cortex®-M family. Additional hardware registers for floating-point operations (FPU) are integrated into specific variants, but will not be used in this lab series

The STM32L476 memory has the following features:

- Flash (1MB) 2 512KB banks read-while-write, proprietary code readout protection
- SRAM 128KB including 32KB with hardware parity check

Example of peripherals :

- General Purpose Innput/output (GPIO) (
- Watchdog : IWDG (Independant Watchdog) et WWDG (Window Watchdog)
- Timers : basics and advances
- I2C communication module
- SPI communication module
- UART communication module
- ADC (Analog to Digital Converter)
- DAC (Digital to Analog Converter)
- And security features (
- …

Peripheral access is managed across multiple bus architectures, including the **AHB (Advanced High-performance Bus)**, **APB1 (Advanced Peripheral Bus 1)**, and **APB2 (Advanced Peripheral Bus 2)**. Care must be taken to explicitly enable the corresponding peripheral clock before accessing any peripheral registers.

Detailed register descriptions and offset mappings for all integrated peripherals are provided in the microcontroller's **Reference Manual (RM0351)**

Embedded microcontroller architectures utilize distinct memory types based on functional requirements: non-volatile memory (**Flash or ROM**) for storing executable program code and read-only constants, and volatile read-write memory (**SRAM**) for temporary runtime variables and stack operations. However, from the programmer's perspective, the Arm® Cortex®-M architecture unifies these physically separate memories into a **single continuous 32-bit address space**, as illustrated in Figure 2:

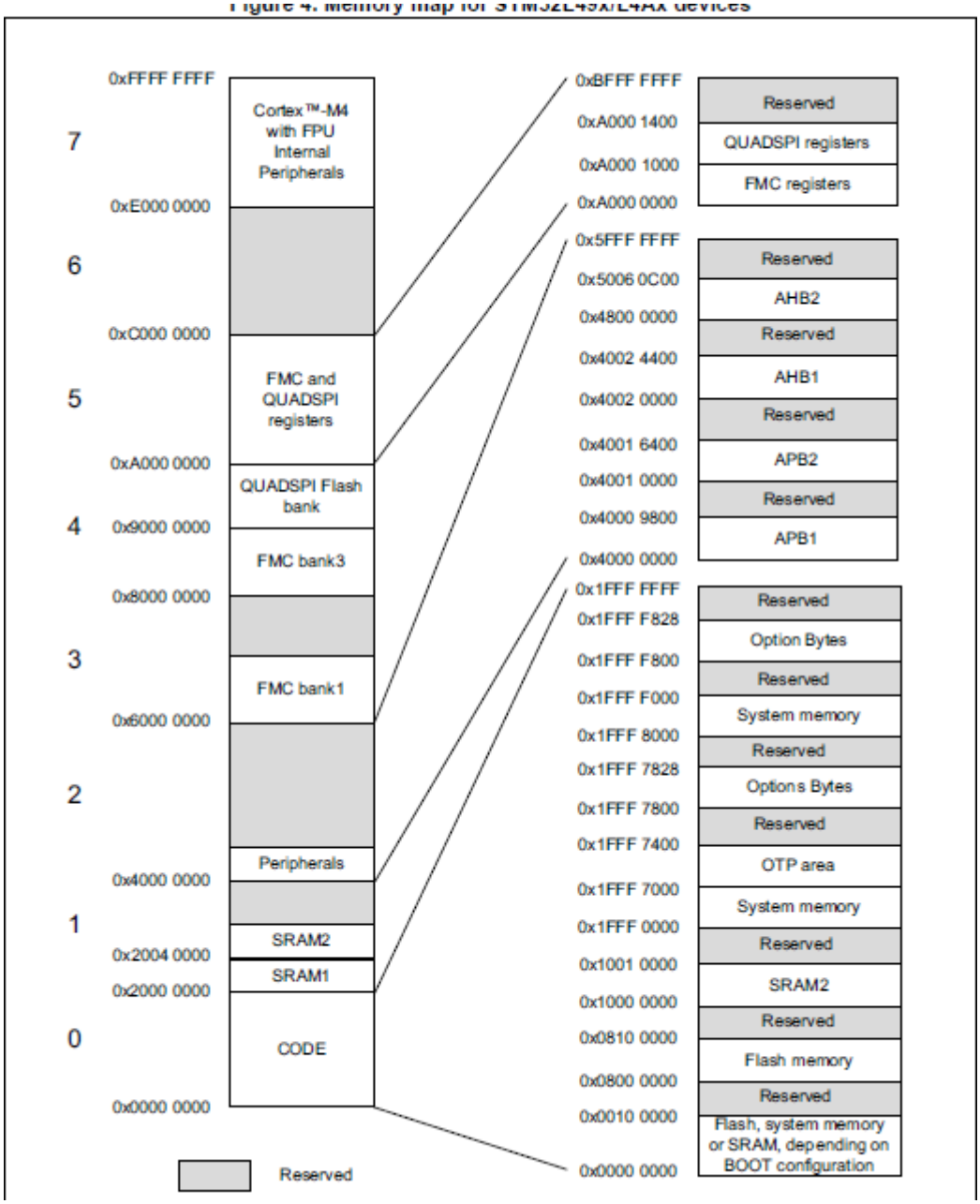


Figure 2 : organization of addressable space

STM32L476xx devices feature up to 128 Kbyte of embedded SRAM. This SRAM is split into two blocks:
The memory can be accessed in read/write at CPU clock speed with 0 wait states.

- 96 Kbyte mapped at address 0x2000 0000 (SRAM1)
- 32 Kbyte located at address 0x1000 0000 with hardware parity check (SRAM2). This block is accessed through the ICode/DCode buses for maximum performance. These 32 Kbyte SRAM can also be retained in Standby mode. The SRAM2 can be write-protected with 1 Kbyte granularity.

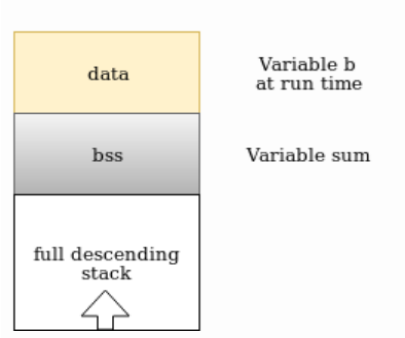


Figure 3 : organization of RAM space

As shown in the memory structure, **SRAM** is partitioned into three distinct sections managed by the linker and runtime startup code:

1. **`.data` Section (Initialized Data):** Contains global and static variables explicitly initialized with non-zero values in C code. This region always begins at the base of SRAM (**`0x20000000`**) and grows upward toward higher addresses.
2. **`.bss` Section (Uninitialized Data):** Reserves space for global and static variables that are uninitialized or initialized to zero. Its starting address immediately follows the end of the `.data` section and grows upward.
3. **Stack Space:** Managed dynamically at runtime for local variables, register saving, and function call frames. The stack grows downward toward lower memory addresses. Its initial top-of-stack address is configured at the end of SRAM based on the total memory capacity (e.g., **`0x20014000`** for an 80 KB SRAM footprint, computed as `0x20000000 + 80 KB`).

Peripherals are designated by name and actually correspond to access to particular registers located in memory addresses starting from 0x4000 0000.

The interrupt vectors start at the address 0x0000 0004 of the program memory. They define the addresses of the subroutines to be executed when an interrupt arrives. For example, the interrupt due to the Reset of the microcontroller is linked to the address of the start of the main program (main). Thus, as soon as the microcontroller is powered on, the reset interrupt is generated, and this forces the CPU to jump to the address of the start of the program to execute it. A table gives the correspondence between the interrupt vectors (the number of interrupts depends on the microcontrollers. ARM allows a maximum of 256 interrupts, but the vast majority of microcontrollers have less interrupt vectors. The STM32L152RE has 81 maskable interrupts + 16 lines of interrupts of the M4 cortex.

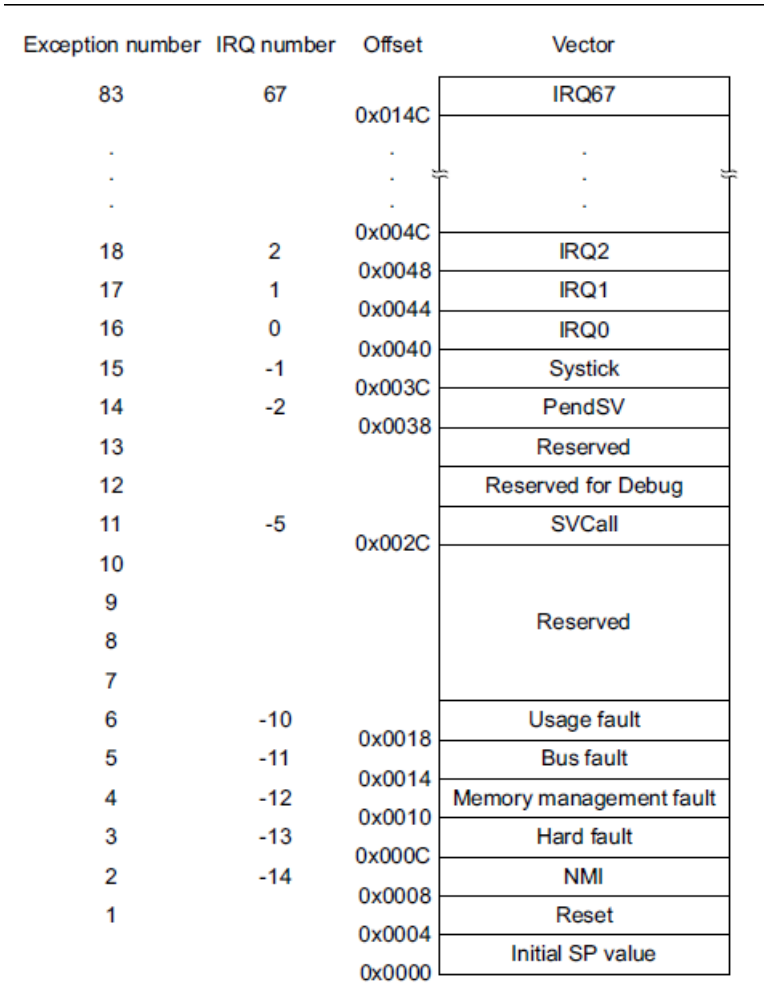


Figure 4 : Table des vecteurs d'interruption

Table 58. STM32L47x/L48x/L49x/L4Ax vector table

Position	Priority	Type of priority	Acronym	Description	Address
-	-	-	-	Reserved	0x0000 0000
-	-3	fixed	Reset	Reset	0x0000 0004
-	-2	fixed	NMI	Non maskable interrupt. The RCC Clock Security System (CSS) is linked to the NMI vector.	0x0000 0008
-	-1	fixed	HardFault	All classes of fault	0x0000 000C
-	0	settable	MemManage	Memory management	0x0000 0010
-	1	settable	BusFault	Pre-fetch fault, memory access fault	0x0000 0014
-	2	settable	UsageFault	Undefined instruction or illegal state	0x0000 0018
-	-	-	-	Reserved	0x0000 001C - 0x0000 0028
-	3	settable	SVCall	System service call via SWI instruction	0x0000 002C
-	4	settable	Debug	Monitor	0x0000 0030
-	-	-	-	Reserved	0x0000 0034
-	5	settable	PendSV	Pendable request for system service	0x0000 0038
-	6	settable	SysTick	System tick timer	0x0000 003C
0	7	settable	WWDG	Window Watchdog interrupt	0x0000 0040
1	8	settable	PVD_PVM	PVD/PVM1/PVM2/PVM3/PVM4 through EXTI lines 16/35/36/37/38 interrupts	0x0000 0044
2	9	settable	RTC_TAMP_STAMP /CSS_LSE	RTC Tamper or TimeStamp /CSS on LSE through EXTI line 19 interrupts	0x0000 0048
3	10	settable	RTC_WKUP	RTC Wakeup timer through EXTI line 20 interrupt	0x0000 004C
4	11	settable	FLASH	Flash global interrupt	0x0000 0050
5	12	settable	RCC	RCC global interrupt	0x0000 0054
6	13	settable	EXTI0	EXTI Line0 interrupt	0x0000 0058
7	14	settable	EXTI1	EXTI Line1 interrupt	0x0000 005C
8	15	settable	EXTI2	EXTI Line2 interrupt	0x0000 0060
9	16	settable	EXTI3	EXTI Line3 interrupt	0x0000 0064
10	17	settable	EXTI4	EXTI Line4 interrupt	0x0000 0068
11	18	settable	DMA1_CH1	DMA1 channel 1 interrupt	0x0000 006C

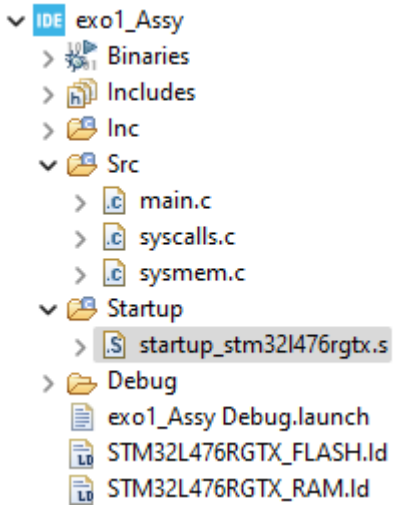
Figure 5 : Interrupt Vector Table with Address

2.2. Assembly language programming

2.2.1. Program skeleton

The µcontroller is programmed in assembly language following the Thumb-2, ARMV7 instructions. The assembler has a complete instruction set and many addressing modes.

The startup program is always written in assembly language. When STM32CubeIDE initializes a project, a startup_stm32l476rgtx.s file is created. An .s file means that the file contains assembly code.



The directives in this file follow the gcc standards. It is not necessary to know these directives for the lab. Another file is provided with instructions where to write the code and declare variables in order to focus only on the assembly code. After creating a project, replace the startup file with the one provided.

The skeleton of the program " stm32l476rgtx.s " provided is intended to clearly define the structure of a program and the different memory areas necessary for the construction of the executable code. It will serve as a basis for writing your programs and includes several areas.

2.2.1.1. Description area

This area is made up of comments and allows you to describe what the program does. It is very important to comment programs, regardless of the programming language, but it is especially true in assembly language, because it is a language that is not very readable.

The syntax of comments is as follows:

```
/* this is a one-line comment */
INST                    // commentaire d'une instruction
```

2.2.1.2. Area to declare symbols

This area allows you to declare symbols to simplify reading the source code. They are designated by a name (also called a label) and do not take up space in memory (unlike constants). During compilation, the label is automatically replaced by the value it designates. This is the equivalent of a "#define" in C.

The syntax is as follows:

```
.equ    name, value
```

For instance :

```
47 /*****************************************************************************
48 *                                                                           *
49 *                  AREA TO DECLARE SYMBOLS                                  *
50 *                                                                           *
51 *****************************************************************************/
52 .equ myvar, 0x87654321
53
54 /*****************************************************************************
55 *                                                                           *
56 *               END   AREA TO DECLARE STMBOLS                               *
57 *                                                                           *
58 *****************************************************************************/
```

Notes :

- Remember to give clear names to simplify the reading of the code. This remark is valid for all types of names that you will define: symbols, variables, constants, subroutines.

- For the readability of your code, use the following convention where [TAB] designates the tab key:

Label :

[TAB]INST [TAB]operand[TAB] // comment

2.2.1.3. Variable declaration area

This area is located in the RAM of the memory. Each variable is declared by a name (or label) and a size in number of bytes or words (1 word = 4 bytes). To simplify the program, we will only declare words. We can also declare tables and character strings.

The syntax is as follows :

Variable name :

.word value

And for an array

Variable Name :

.word value1, value2, value3

A few declaration examples:

```
/*****************************************************************************
*                                                                            *
*                  AREA TO DECLARE INITIALIZED VARIABLES                     *
*                                                                            *
*****************************************************************************/
.section .data

CPT:
    .word 10

varindata:
    .word 0x12345678, 0x0A, 0xDEADBEEF
    .string "hello word"

decalage:
    .word 0x3

/*****************************************************************************
*                                                                            *
*              END   AREA TO DECLARE INITIALIZED VARIABLES                   *
*                                                                            *
*****************************************************************************/
```

Similarly, it is also possible to declare uninitialized variables in the bss section.

2.2.1.4. Constants declaration area

This area is located in the program memory area (Flash). Each constant is declared by a name (or label) and a value. We can also declare tables of constants

```
/*****************************************************************************
*                                                                            *
*                   AREA TO DECLARE CONSTANTS                                *
*                                                                            *
*****************************************************************************/
.section .text
varconstant:
    .word 0xDEADBEEF


/*****************************************************************************
*                                                                            *
*                END   AREA TO DECLARE CONSTANTS                             *
*                                                                            *
```

To simplify the program, only declare words that correspond to 32-bit length variables.

The syntax is as follows :
Variable name :
        .word value
And for an array
Variable Name :
        .word value1, value2, value3

2.2.1.5. Subprogram declaration area

This area is also in program memory. It will contain all the subroutines you define.

The syntax of a subroutine is as follows:

```
name :                          ; name of the subprogram
        INST1
        INST2
        …
        bx lr                   ; instruction end of subprogram
```

This structure assumes that the return address has been saved in the LR register.

```
/*****************************************************************************
*                                                                            *
*               AREA TO DECLARE SUBPROGRAMS                                  *
*                                                                            *
*****************************************************************************/
dosomething:
    movs r0,#0xAB
    bx lr

/*****************************************************************************
*                                                                            *
*               END    AREA TO DECLARE SUBPROGRAMS                           *
*                                                                            *
*****************************************************************************/
```

2.2.1.6. Main Program

This main program is located in the program memory. **It is the starting point of the program**.

Upon system reset, the microcontroller executes its hardware startup sequence by automatically loading the initial **Main Stack Pointer (MSP)** value from the first entry of the vector table (address `0x00000000` / `0x08000000`) and the initial **Program Counter (PC)** value from the second entry (`Reset_Handler`). This initialization code is automatically generated by the IDE during project setup and requires no manual modification.

```
  .section .text.Reset_Handler
  .weak Reset_Handler
  .type Reset_Handler, %function
Reset_Handler:
  ldr   r0, =_estack
  mov   sp, r0          /* set stack pointer */

/* Copy the data segment initializers from flash to SRAM */
  ldr r0, =_sdata
  ldr r1, =_edata
  ldr r2, =_sidata
  movs r3, #0
  b LoopCopyDataInit
```

After the initialization, its syntax is as follows :

```
Main :
        Instructions   //  initializations of various peripherals (clocks, timer, GPIO…)
```

boucl …
…
b boucl ; loop

```
/**************************************************
*
*               Main Program
*
/**************************************************

main:

LoopForever:

    b LoopForever
```

The "b boucl" instruction makes a jump in the main program. It therefore loops infinitely. This must always be the case. Indeed, a microcontroller is generally embedded in a system that operates continuously as long as it is powered on. The main program must therefore ensure that it never returns thanks to an infinite loop.
In general, the "infinite" loop is performed by a jump to a position that is after the initialization phase.

2.2.1.7. Interrupt Subroutine Declaration Area

This area is located in program memory. It will contain all the interrupt routines you define.

These interrupt routines are the routines to be executed when an interrupt (IT) from a device or port of the µcontroller occurs.

The syntax of an interrupt subroutine is:

```
/*****************************************************************************
*                                                                            *
*              Area to Declare interrupt sub-programs                        *
*                                                                            *
/*****************************************************************************/
testfunc:
// Instructions
    bx lr

/*****************************************************************************
*                                                                            *
*             END  Area to Declare interrupt sub-programs                    *
*                                                                            *
/*****************************************************************************/
```

The name of the interrupt programs is imposed by ARM for portability purposes between all manufacturers of microcontrollers based on the Cortex M core. When creating a project, functions are already created in the weak aliases section

```
/*******************************************************************************
*
* Provide weak aliases for each Exception handler to the Default_Handler.
* As they are weak aliases, any function with the same name will override
* this definition.
*
*******************************************************************************/

    .weak   NMI_Handler
    .thumb_set NMI_Handler,Default_Handler
```

All interrupts call a default_Handler.

For simplification, to call an interrupt program, change the default_handler to the name of the function that should be called by the interrupt.

```
.weak   EXTI15_10_IRQHandler
.thumb_set EXTI15_10_IRQHandler,testfunc
```

2.2.1.8. Interrupt Vector Declaration Area

This area is located in program memory. Interrupt vectors are actually the links between the interrupt itself and the corresponding interrupt subroutine.

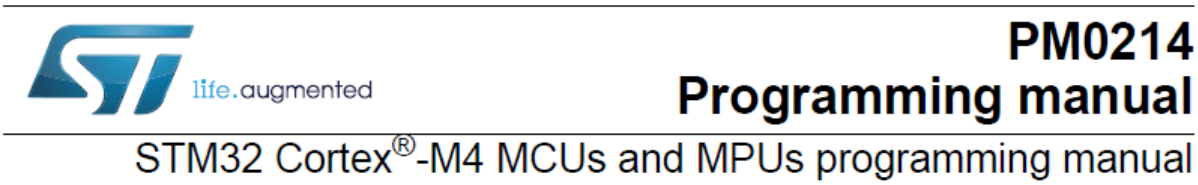
```
/*******************************************************************************
*
* The STM32L152RETx vector table.  Note that the proper constructs
* must be placed on this to ensure that it ends up at physical address
* 0x0000.0000.
*
*******************************************************************************/
  .section .isr_vector,"a",%progbits
  .type g_pfnVectors, %object
  .size g_pfnVectors, .-g_pfnVectors

g_pfnVectors:
  .word _estack
  .word Reset_Handler
  .word NMI_Handler
  .word HardFault_Handler
```

These names are imposed by ARM. Nothing should be changed in this area

2.3. Cortex M Instructions

All instructions are presented in the STM32 L476 Programming Manual.



The explanation of instruction encoding is explained in the ARMv7-M Architecture Reference Manual.

2.4. STM32 addressing modes

The STM32 family has various addressing modes. Not all instructions necessarily accept all addressing modes, so you should check in the help which mode is appropriate. During your practical work, you will often have to manipulate values in decimal, binary but also in hexadecimal. In assembler, this translates as follows:

- to code the value 10 in decimal, we will write: **10**
- to code the value 10 in binary, we will write: **0b00001010**
- to code the value 10 in hexadecimal, we will write: **0x0A**

2.4.1. Immediate Mode

To load a register with a fixed value, this value can be incorporated into the instruction. In this case, the value is stored in the program and there is no need to give the address of the data to be read, the address to point to will be calculated directly by the compiler.

For instance :

```
Ndoigts     EQU   10
Hexa        EQU   0x10
Mask        EQU   0b10100011
            …
            LDR   r0,=Ndoigts    ; r0 is loaded with the value 10
            LDR   r0,#Hexa       ; r0 is loaded with the hexadecimal value 10h
            LDR   r0,#Mask       ; r0 is loaded with the binary value 1010 0011 (0xA3h)
```

2.4.2. Register Mode

In this mode, the address where the value we are interested in, is stored in the rn register. The value is read from memory and stored in the rd register.

Syntax : LDR rd,[rn]

Example :

```
LDR   r1,[r0]        ; r0 is loaded with the value of the word located at the address stored in r0
```

2.4.3. Register Mode Register with immediate offset

In this mode, the address where the value we are interested in, is stored in the rn register incremented by the offset. The value is read from memory and stored in the rd register.

Syntax : LDR rd,[rn,#offset]

Example for reading in a table

```
varindata2:
    .word 0xFFFFFFFF, 0xDEADBEEF, 0xA

movs r0, #0b1010
ldr r0, =varindata2
ldr r1, [r0,#4]
```

After execution of the code, the value stored is r1 is equal to 0xDEADBEEF

2.4.4. Register Mode with Offset Register

Syntax : LDR rd,[rn,rm]

```
varindata2:
    .word 0xFFFFFFFF, 0xDEADBEEF, 0xA
main:
movs r0, #0b1010
ldr r0, =varindata2
ldr r2, =0x4
ldr r1, [r0,r2]
```

At the end of the code, the value stored in r1 is 0xDEADBEEF

2.4.12 Instructions

To find the instructions, refer to the course and especially to the Programming Manual which explains all the instructions accepted by the STM32L476.

2.5. Development set

The development board is the NUCLEO-L476RE. All the useful documentation is available on the STMicroelectronics website

https://www.st.com/en/evaluation-tools/nucleo-l476rg.html

The development software used is STM32CubeIDE which has the advantage of running on Windows, MAC and Linux. It is free and has no memory size limit.

2.5.1. Project Creation

A document has been written to help you create an STM32 project in the STM32CubeIDE development environment (IDE). Refer to this document for each new program. You will find it in the appendix.
Writing assembly and C code is done in the IDE with a syntax coloring editor. Compilation and linking are done in the same environment as well as debugging. Note that the STM32CubeIDE does not have a simulator and the board is therefore necessary to test a program.

2.5.2. Using the Debugger

The STM32CubeIDE has many features that cannot be taught all. However, Appendix 3 and Chapter 3 show you the essential windows to use to properly develop a program. When compiling, read the error messages. When debugging, use all these tools at your disposal to fully understand the execution of your code.

The teachers present with you are also there to help you get to grips with this environment, so don't hesitate to ask questions.

Chapter 3. First Steps

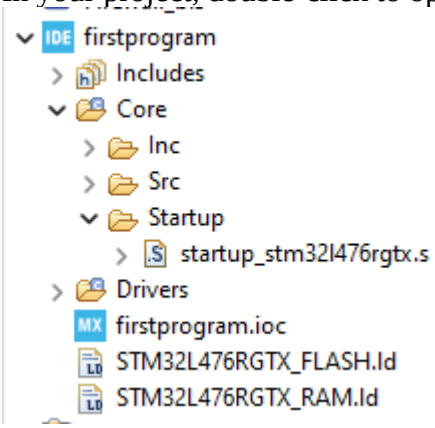
3.1. Introduction

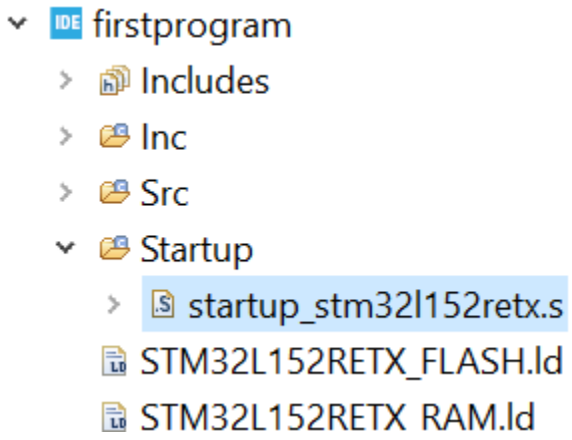
The initial phase of this practical exercise involves configuring the development workspace and establishing the project structure required to assemble, link, and debug your first Arm assembly application

First, follow the steps described in Appendix 2 "Creating an STM32 project in assembler".

After creating a project, we will show you how to write a first program and how to use the debug windows.

In your project, double-click to open the startup_stm32l476rgtx.s file





In your project, double-click to open the startup_stm32l152retx.s file

```
76 /*******************************************************************************
77 *                                                                              *
78 *                  AREA TO DECLARE INITIALIZED VARIABLES                       *
79 *                                                                              *
80 *******************************************************************************/
81 .section .data
82
83
84
85 /*******************************************************************************
86 *                                                                              *
87 *              END   AREA TO DECLARE INITIALIZED VARIABLES                     *
88 *                                                                              *
89 *******************************************************************************/
```

To declare a 32-bit variable (for the sake of simplification, only declare 32-bit variables for the assembler labs). The syntax is as follows:

Variable name:
.word value of the variable

For example, to declare the variable named ‘myVariable’ initialized to 2, the code is written as below

```
76 /*******************************************************************************
77 *                                                                              *
78 *                  AREA TO DECLARE INITIALIZED VARIABLES                       *
79 *                                                                              *
80 *******************************************************************************/
81 .section .data
82
83 maVariable:
84     .word   2
85
86
87 /*******************************************************************************
88 *                                                                              *
89 *              END   AREA TO DECLARE INITIALIZED VARIABLES                     *
90 *                                                                              *
91 *******************************************************************************/
```

As an example, the program below retrieves the address of the variable myVariable at the beginning of the program and then subtracts 1 in the infinite loop.

Write the following program:

```
main:

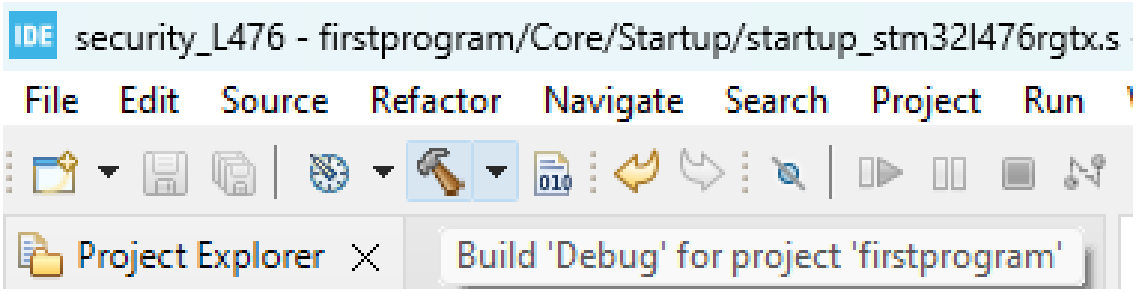
ldr r0, =maVariable // récupère l'adresse de la variable r0
ldr r1, [r0] // mets la valeur pointée par r0 dans r1

LoopForever:
// soustrait 1 à r1 en mettant à jour le registre xPSR
    subs r1,#1

    b LoopForever

 .size Reset_Handler, .-Reset_Handler
```

Compile the program



Compilation provides information about the program's memory size, the size of SRAM used, and possible errors that prevent compilation.

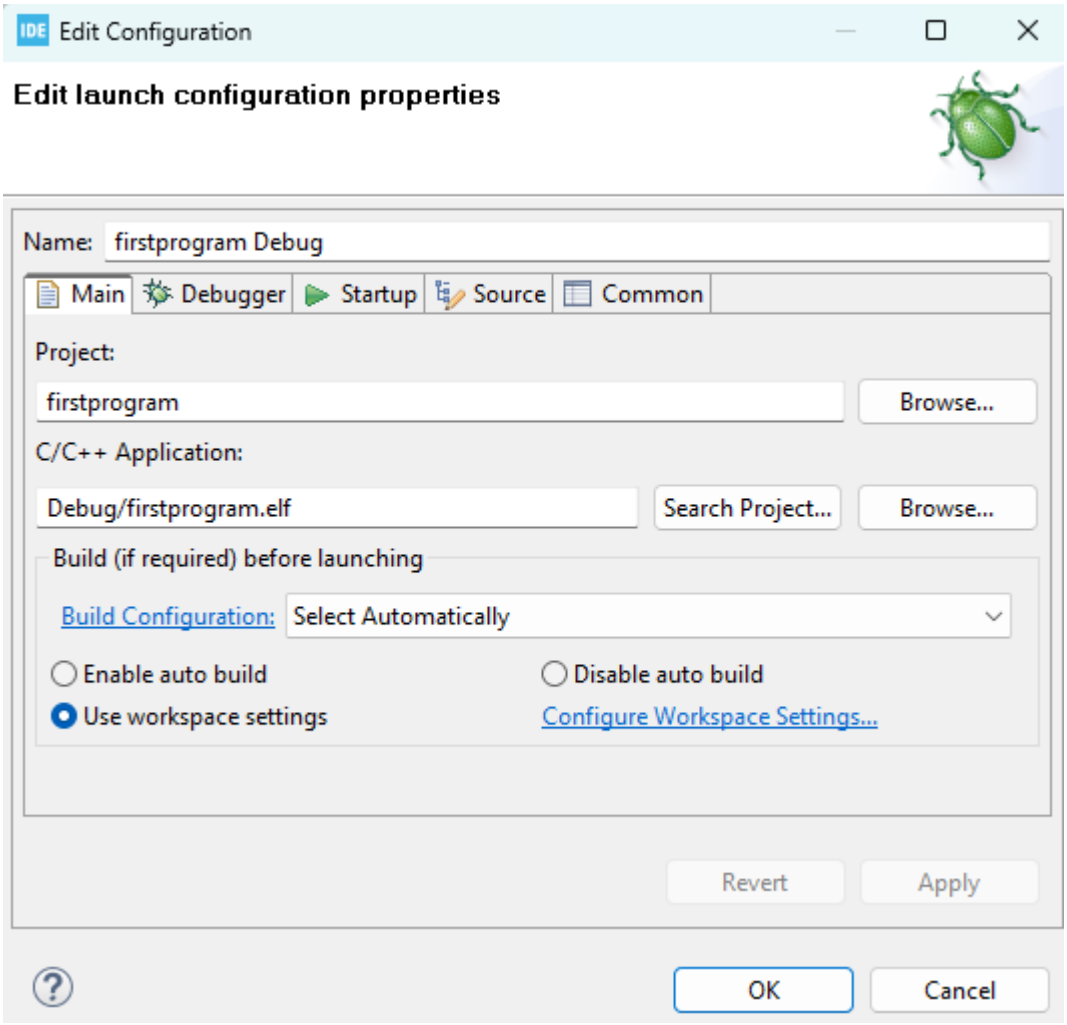
```
Finished building target: firstprogram.elf

arm-none-eabi-size  firstprogram.elf
arm-none-eabi-objdump -h -S firstprogram.elf  > "firstprogram.list"
   text    data     bss     dec     hex filename
    808      16    1568    2392     958 firstprogram.elf
Finished building: default.size.stdout

Finished building: firstprogram.list

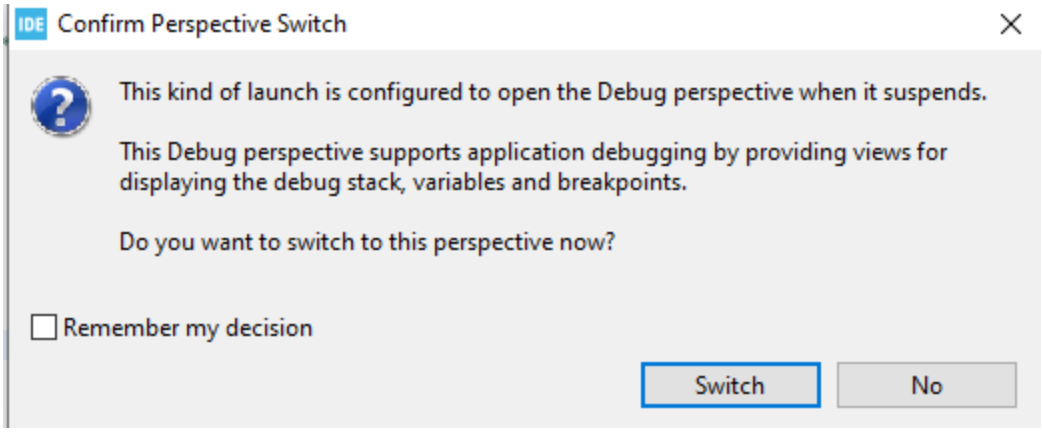

14:22:40 Build Finished. 0 errors, 0 warnings. (took 745ms)
```

Run the program in debug mode. The first time, you are asked to confirm the debug configuration.



Click OK

When the IDE asks to switch to the debug perspective, click switch.



The program stops at the first line of code in main

```
192 main:
193
194 ldr r0, =maVariable // get address of maVariable into r0
195 ldr r1, [r0] // store the value pointed by address in r0 into r1
196
197
198 LoopForever:
199     subs r1, #1
200
201     b LoopForever
202
203 .size   Reset_Handler, .-Reset_Handler
204
```

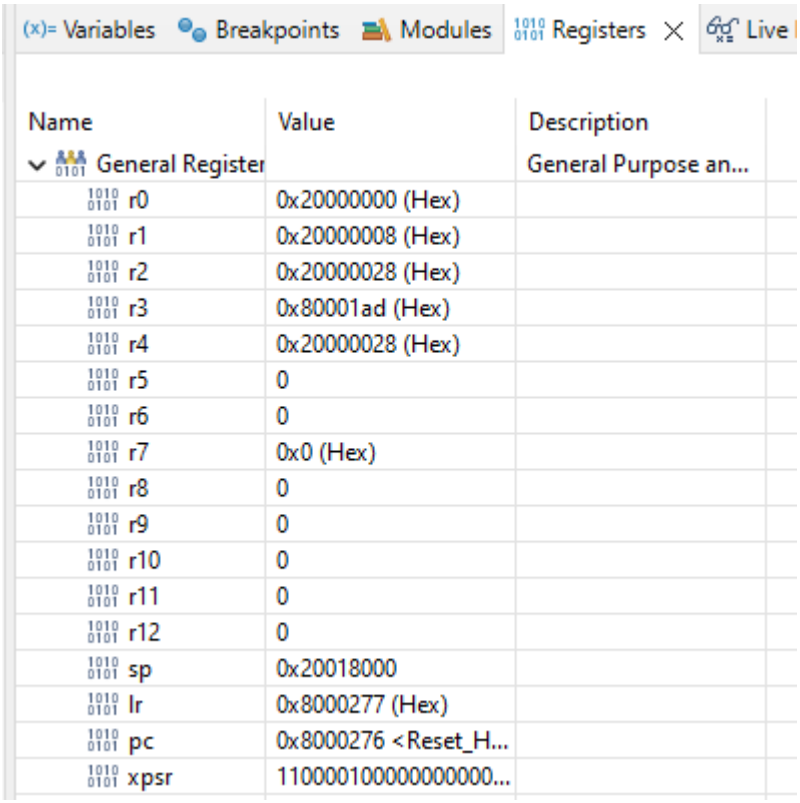
Open the disassembly window, which allows to see the written code, the compiled code, the encoding and the address of the instructions.

```
170                     bl  main
08000272: 00 f0 00 f8   bl      0x8000276 <Reset_Handler+54>
194                   ldr r0, =maVariable // get address of maVariable into r0
08000276: 08 48         ldr     r0, [pc, #32]   ; (0x8000298 <LoopForever+30>)
195                   ldr r1, [r0] // store the value pointed by address in r0 into r1
08000278: 01 68         ldr     r1, [r0, #0]
199                     subs r1, #1
0800027a: 01 39         subs    r1, #1
201                     b LoopForever
```

Inspecting the Disassembly View reveals that execution is currently halted at Flash memory address 0x08000276. Note that the instruction located at 0x08000276 has not yet been executed. In the Arm® Cortex®-M architecture, the Program Counter (PC / R15) always points to the next instruction queued for execution, rather than the instruction that has just completed.

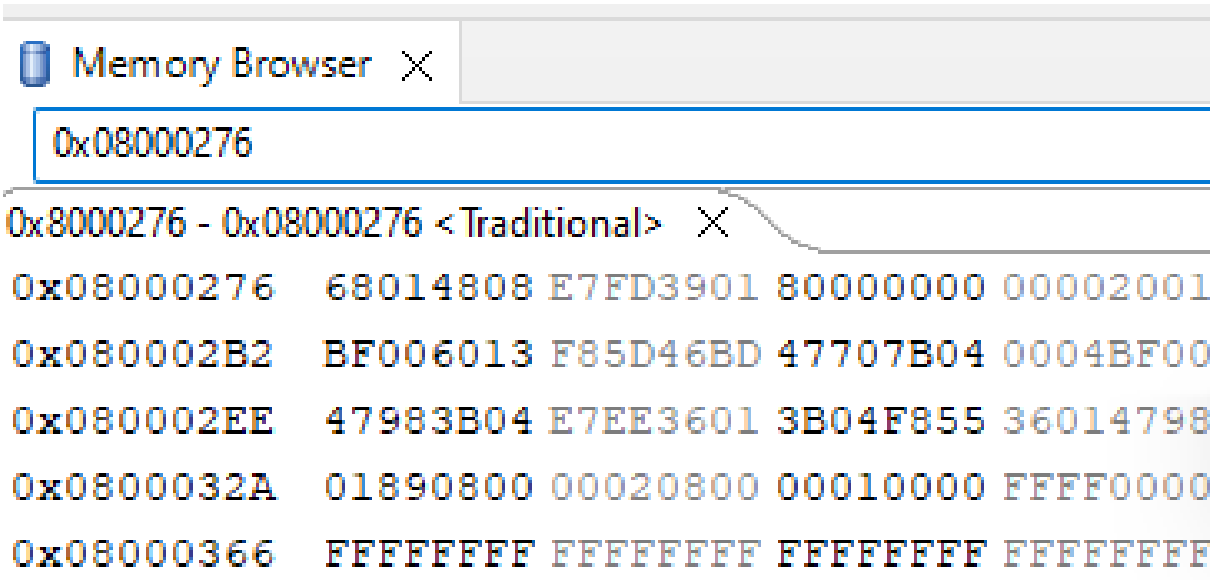
Note: the address can be different depending on compilators and set up.

Open the registers window



The value of the Program Counter is indeed 0x08000276. The next instruction to execute will be: ldr r0, =myvariable

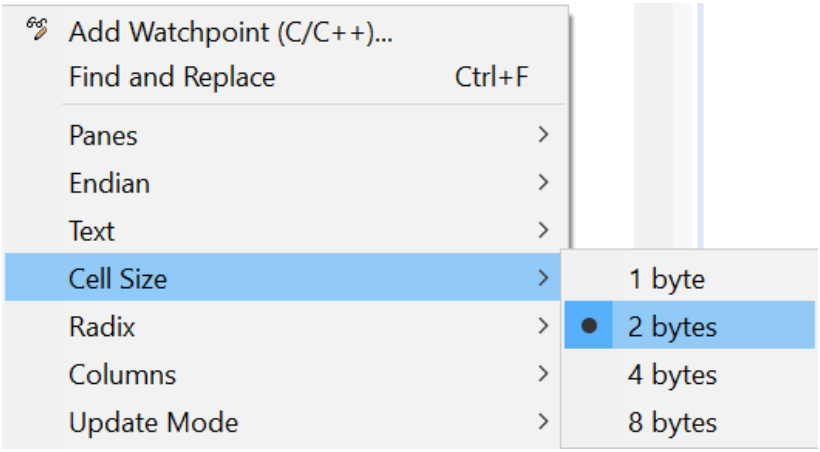
Open the Memory Browser window and bring up the memory at address 0x08000276



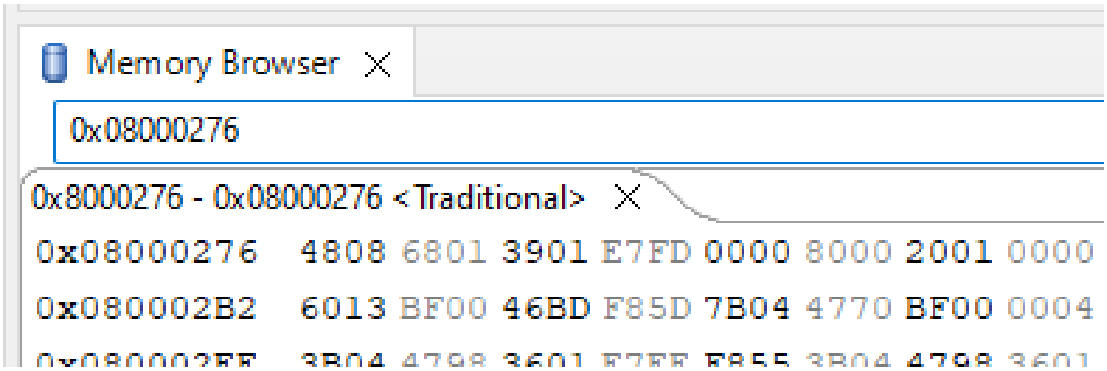
By default, the IDE renders memory contents in 32-bit (4-byte) blocks. However, because the Arm Thumb-2 instruction set predominantly utilizes 16-bit wide opcodes (mixed with select 32-bit instructions), inspecting instruction encoding is significantly clearer when using a 16-bit display format.

Procedure to reformat the Memory View:

1. Right-click anywhere within the **Memory Browser** / **Memory View** window.
2. Select **Format** $\rightarrow$ **Cell Size** (or **Size**).
3. Choose **2 Bytes** (16-bit half-word layout).

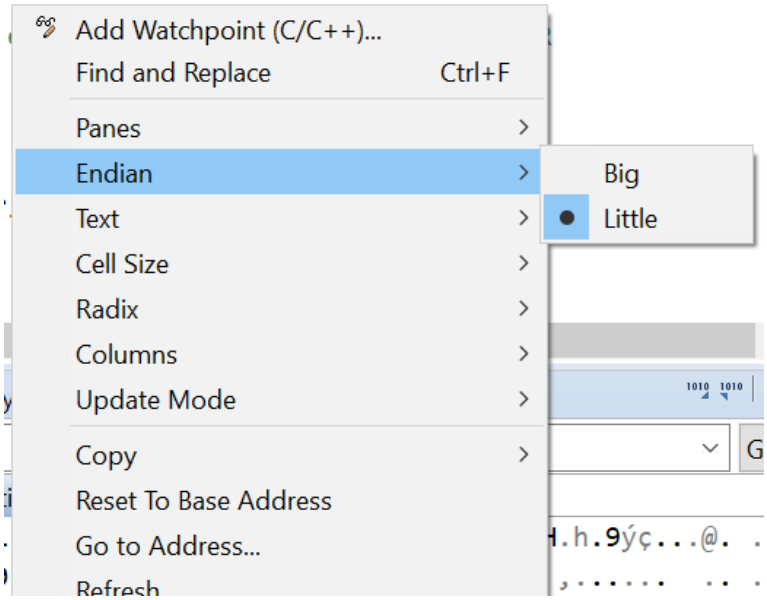


The window is then organized as follows

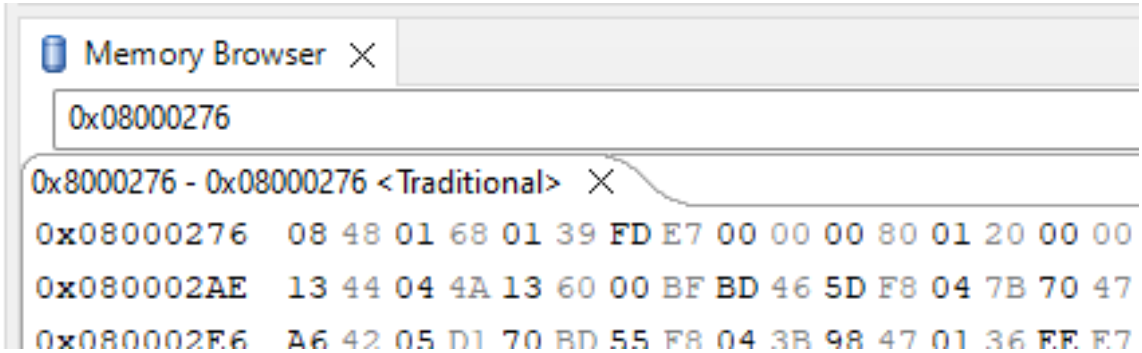


We see that at address 0x08000276, in memory, we have the encoding 0x4808 while the disassembly window gives 0x0848.

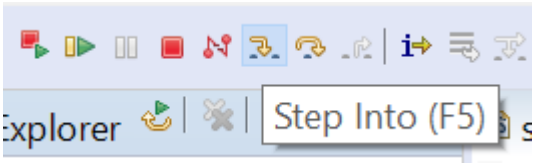
The STM32L476 is configured in little endian.
Place the cursor on the Memory Browser window, right-click and select little endian.
The display is indeed in little endian. We see that at the address 0x08000276, in memory, we have the encoding 0x4808 while the disassembly window gives 0x0848.



If we switch to a byte presentation, we can clearly see that we have 0x08 then 0x48 in memory

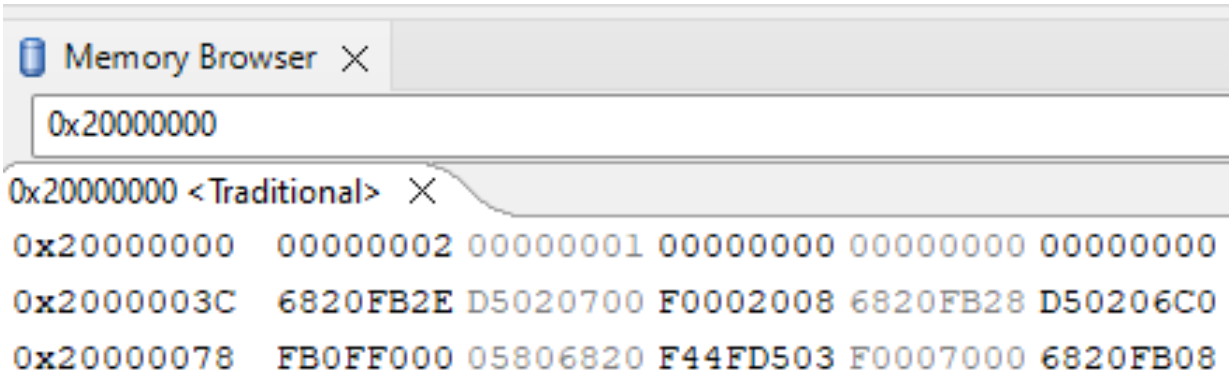


To execute an instruction, step into it by clicking on the arrow icon

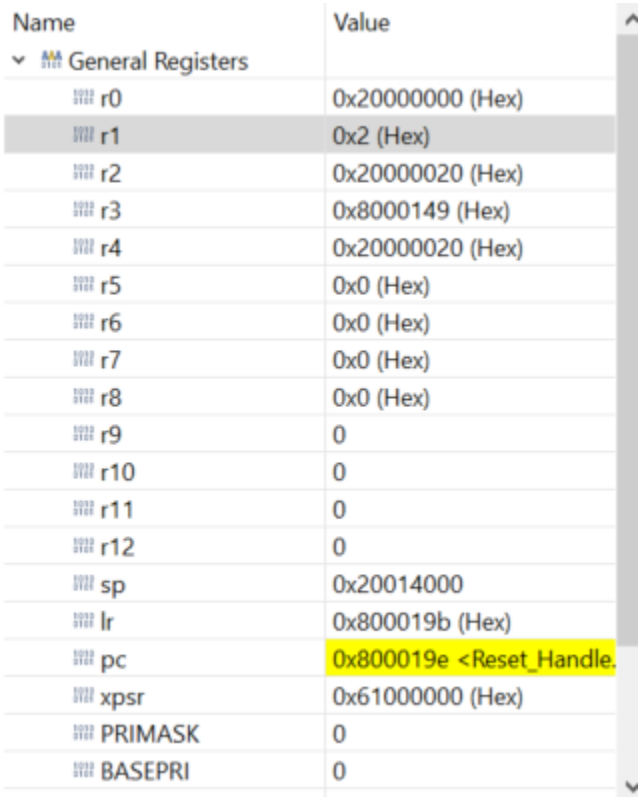


The address of myVariable is copied to r0. The value of r0 is 0x20000000.

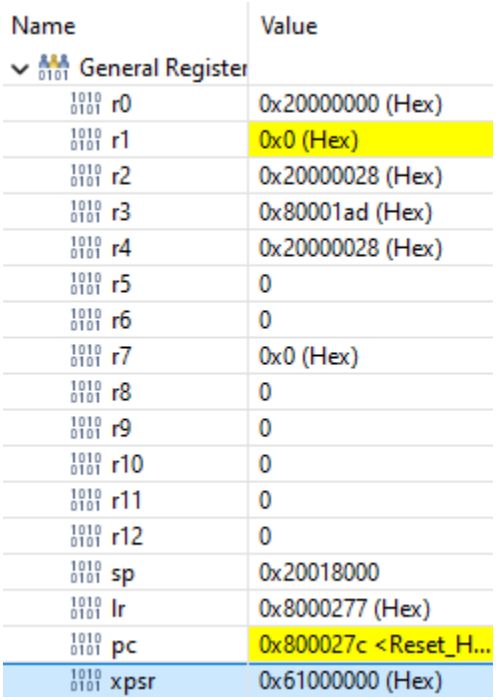
Using the memory browser, go to address 0x20000000. The value 2 is stored at this memory location.



To retrieve the value 2, execute the instruction using step into. The value of register r1 is then 2; note that the PC also changes.

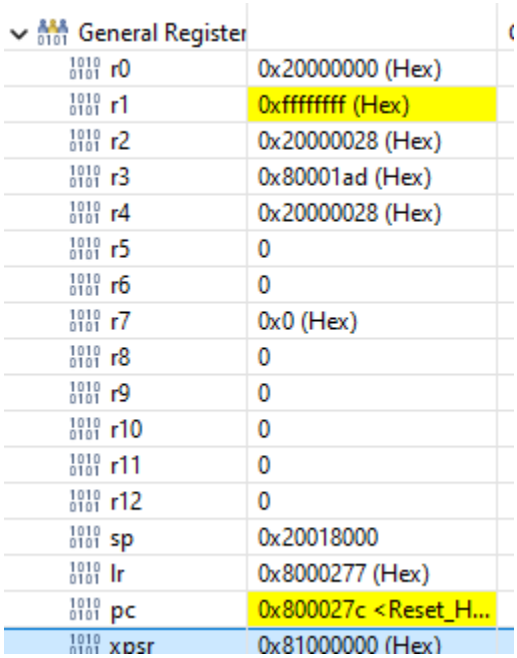


Execute the program step by step. When r1 is 0, xpsr is 0x6100 0020
0x6 = 0b0110 and corresponds to the flags N, Z, C and V



Name	Value
General Register	
r0	0x20000000 (Hex)
r1	0x0 (Hex)
r2	0x20000028 (Hex)
r3	0x80001ad (Hex)
r4	0x20000028 (Hex)
r5	0
r6	0
r7	0x0 (Hex)
r8	0
r9	0
r10	0
r11	0
r12	0
sp	0x20018000
lr	0x8000277 (Hex)
pc	0x800027c <Reset_H...
xpsr	0x61000000 (Hex)

Continue the program to get 0xFFFF FFFF in r1. Check how xPSR was changed. Go back to the course to understand its meaning.



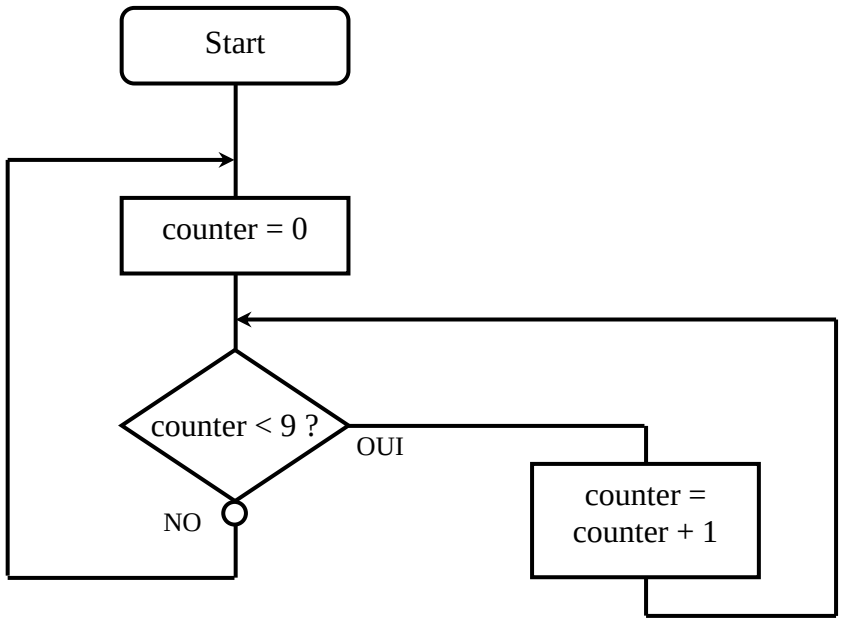
General Register	
r0	0x20000000 (Hex)
r1	0xffffffff (Hex)
r2	0x20000028 (Hex)
r3	0x80001ad (Hex)
r4	0x20000028 (Hex)
r5	0
r6	0
r7	0x0 (Hex)
r8	0
r9	0
r10	0
r11	0
r12	0
sp	0x20018000
lr	0x8000277 (Hex)
pc	0x800027c <Reset_H...
xpsr	0x81000000 (Hex)

To properly debug programs, you will need to follow a similar approach to find your errors. Now it's your turn.

3.2. Construction of a modulo 10 counter

This exercise consists of making a program that counts from 0 to 9 and then starts again at 0. This program loops indefinitely. The counter will be stored in a variable named "count" that you will have to define in the variables area. You will use a repetitive sequence of the type "while ... do".

The following flowchart illustrates the architecture of this program:



You must :

- Write the program following the flowchart above.
- Compile the program.
- Test the program with the debugger in step-by-step mode to verify its proper functioning.
- View and record the evolution of the PC, PSR, R0, R1 registers..
- View and record the evolution of the "account" variable. At what memory address is it located?

Step-by-step execution of the "modulo 10 counter" program:

The values of the registers and the variable correspond to the state before the instruction was executed.

Program Counter PC	Register R0	Register R1	Flags N Z C V	Variable counter	INSTRUCTION	REMARKS

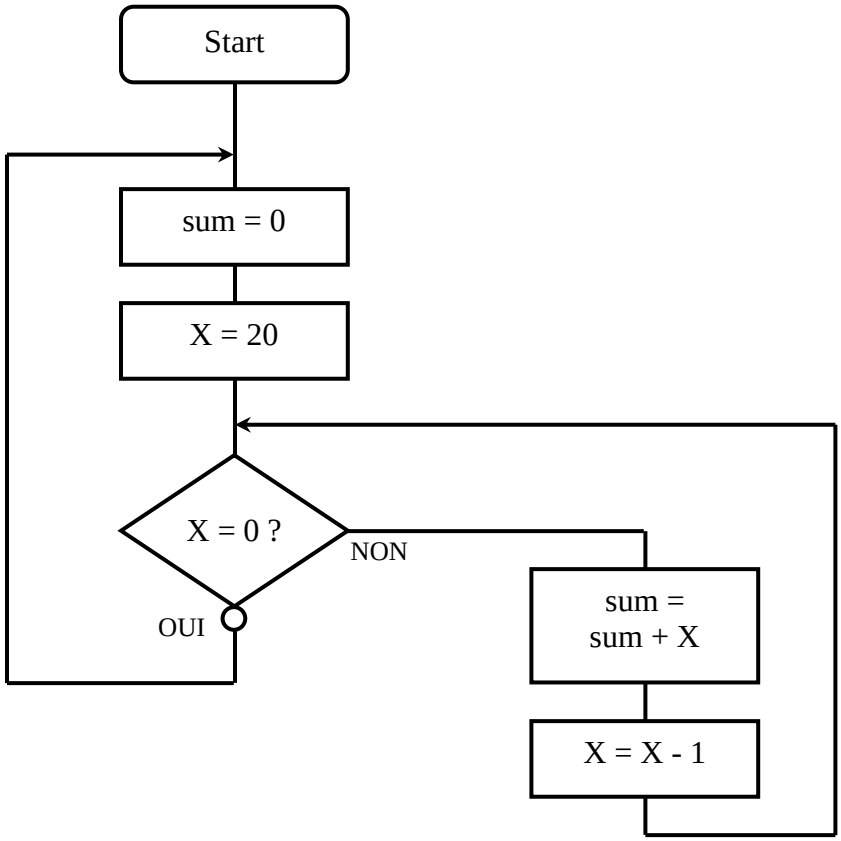
3.3. How to design a for-loop?

The objective of this exercise is to construct a software loop in Arm assembly that replicates the structure and behavior of a high-level language `for` loop (e.g., `for (int i = 0; i < N; i++)`).

To do this, you are asked to calculate the sum of the first 20 integers (1 to 20).

The sum will be stored in a one-byte variable named "sum" that you will have to define in the variables area. However, the loop index "X" will not be declared as a variable.

The following flowchart illustrates the architecture of this program:



You must :

- Calculate the expected final sum to judge the functioning of the program.
- Write the program by respecting the flowchart above.
- Test the program with the debugger in step-by-step mode to verify its proper functioning.
- View the "disassembler" window..
- View the "memory" window.
- Locate in the memory, the "sum" variable and the program.

3.4. Create and call subroutines

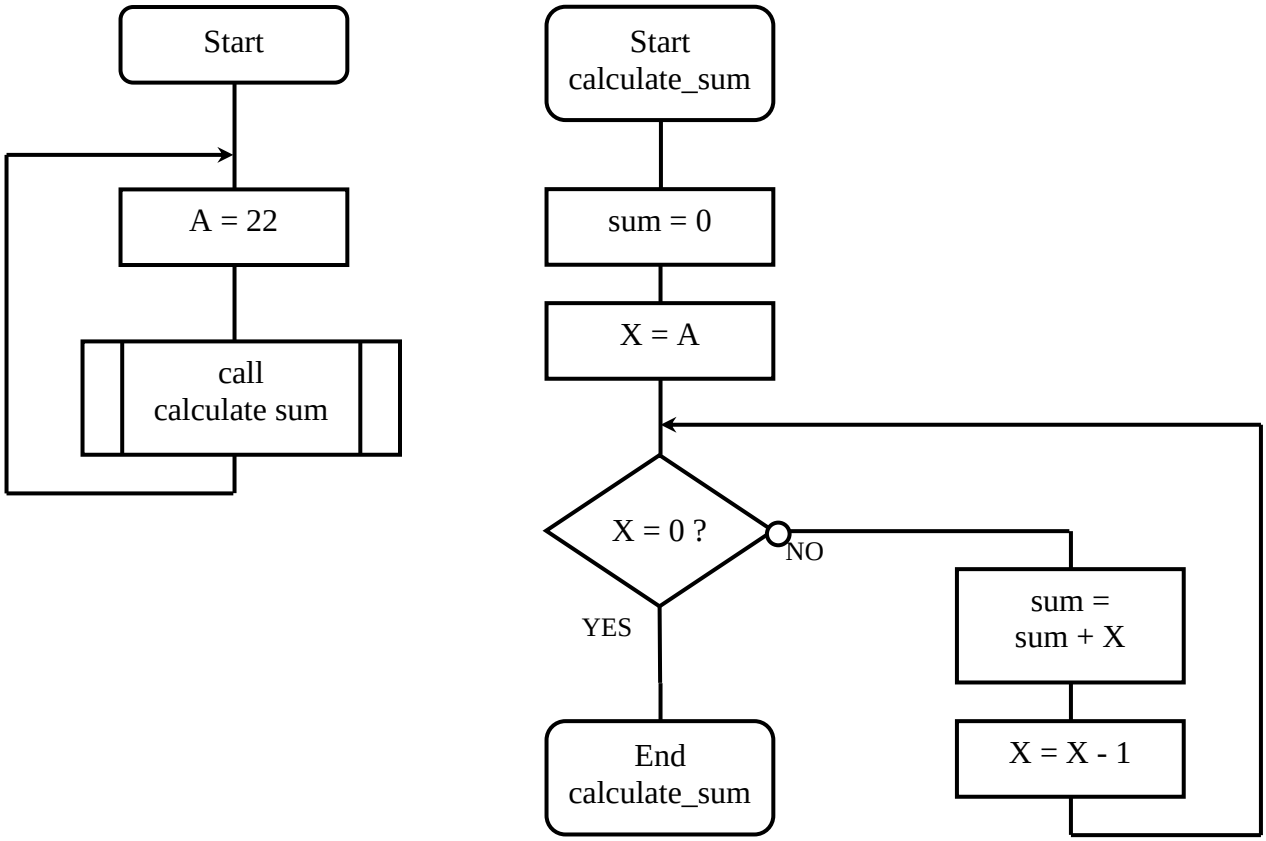
As in high-level languages, assembler allows calls to be made to subprograms (procedures or functions), making a program more readable and more efficient.

This exercise consists of repeating the previous exercise of calculating the sum of the first 20 integers (in a new directory), and creating the subroutine "calculate_sum", which will take as input, in the register r0, the number of integers to add (we are no longer limited to 20) and will calculate the sum which it will return in the register r0. The main program must then store the result in the variable "sum".

This exercise consists of repeating the previous exercise of calculating the sum of the first 20 integers (in a new directory), and creating the subroutine "calculate_sum", which will take as input, in the register r0, the number of integers to add (we are no longer limited to 20) and will calculate the sum which it will return in the register r0. The main program must then store the result in the variable "sum".

The main program will perform the following operations:
- put a number in r0 (number of integers that we want to add),
- call "calculate_sum" with the **bl** instruction.

The following flowchart illustrates the architecture of this program:



You must :

- Write the program following the flowchart above.

- Test the program with the debugger in step-by-step mode to verify its proper functioning.
- Note the evolution of the PC and LR registers.

Call and return of subroutine "calculate_sum" ::

The values of the registers and the variable as well as the contents of the stack correspond to the state **before** the execution of the instruction.
Only the data in the stack at addresses higher than the Stack Pointer are relevant to your program.

Program Counter **PC**	Link Register **LR**	Reg **R0**	Reg **R1**	Condition Flags **N Z C V**	VARIABLE **sum**	INSTRUCTION	Notes

3.5. what is the stack used for ?

This exercise consists of repeating the previous exercise to illustrate the use of the stack with the following constraints:

calcul_sum is called in the infinite loop. In the register r0, you will store the number of times that calcul_sum is called. This means r0 must be initialized to 0 before the infinite loop. R0 is used to pass the parameter to the function and return the sum.

You have to

- using the debugger, note the value of r0 before and after calling "calcul_sum".
- specify the method to keep the number of times that calcul_sum is called?
- test the modified program with the debugger in step-by-step mode to verify that it works correctly.
- note the evolution of the contents of SP, R0 and the memory at the address pointed to by SP

Note: for the rest, in each subroutine, you will make sure to save all the registers used if necessary. ARM provides CMSIS (Cortex Microcontroller Software Interface Standard) standards for passing parameters to functions and saving registers. We will not study these standards in this lab, but for a program in the professional environment, you will have to respect them.

Stack usage :

The values of the registers and the contents of the stack correspond to the state before the instruction was executed. Only the data in the stack at addresses higher than the Stack Pointer are relevant to your program.

Program Counter **PC**	Stack Pointer **SP**	**Stack**								Reg **R0**	Reg **R1**	Flasgs Status **N Z C**	INSTRUCTION	NOTES

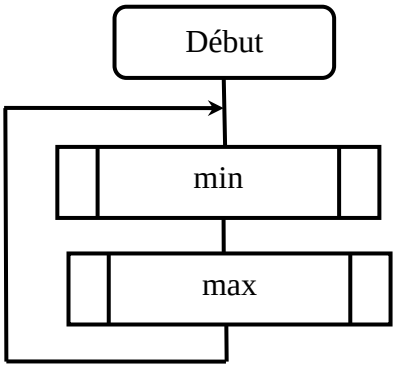
3.6. Using tables

In high-level languages, arrays are widely used to solve many problems. So we will see how to use arrays in assembler with the STM32 family.

This exercise consists of making a program to extract minimum and maximum on an array of unsigned bytes. To do this, you will declare an array of 8 words, as below.

tab[0]	25
tab[l]	4
tab[2]	2
tab[3]	15
tab[4]	16
tab[5]	101
tab[6]	33
tabl[7]	3

You will then create two subroutines "min" and "max" which search for the minimum and maximum values in the array respectively. The results will be stored in the variables "minimum" and "maximum" to be declared.



You have to :

- Detail the flowchart of the “min” and “max” subprograms

When creating a program in a new language, it is interesting to gradually integrate subroutines.

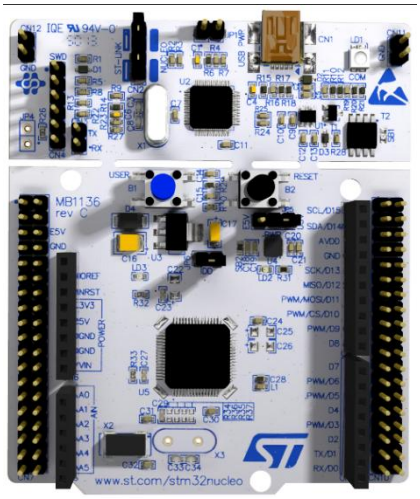
- Write the subroutine "min".
- Test the partial program.
- Write the subroutine "max" based on the subroutine "min".
- Test the complete program.
- Option: Create a single subroutine that returns the min and max

Notes:
- Testing a program involves testing all branches of the program and subroutines.
- Remember to use all the debug windows offered by the STM32CubeIDE

Chapter 4. Development board

Now that you have completed the previous exercises, you have acquired the basics of assembly programming on the STM32 family, and you are able to create a program and debug it with the STMicroelectronics STM32CubeIDE development environment.

We will now briefly introduce the NUCLEO-L476 development board developed by STMicroelectronics to test the functionalities of a microcontroller.

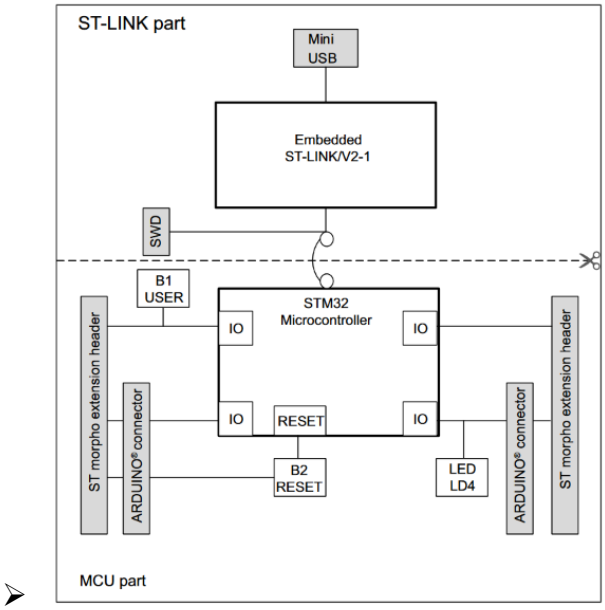


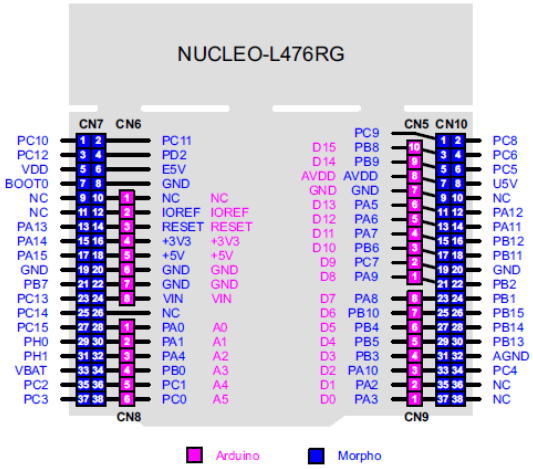
This board can also be useful for a project. It allows you to use the different components and functionalities of the µcontroller.

4.1. Available Peripherals on the nucleo board

The following components are available on the board :

- 1 LED that lights up with a state "1", connected to pin PA5
- 1 push button which gives a '0' state when pressed, connected to pin PC13.
- 1 Reset push button
- An Arduino connector and a Morpho connector that provide access to the microcontroller pins and allow easy connection of external components.
- A USB connection for debugging and programming with SWD connector





STMicroelectronics provides the schematic of the board which allows you to see how the pins of the microcontroller are connected to the board

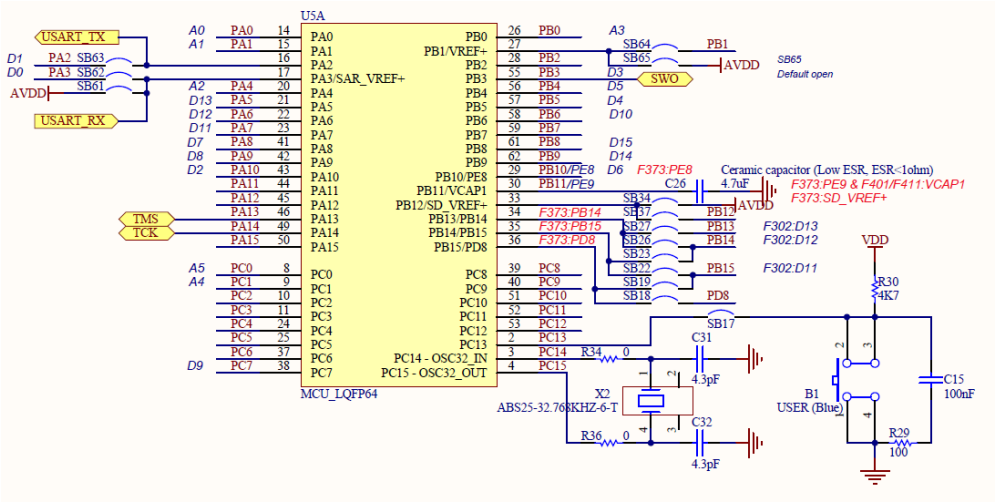


Figure : Connections STM32L476RG Nucleo board

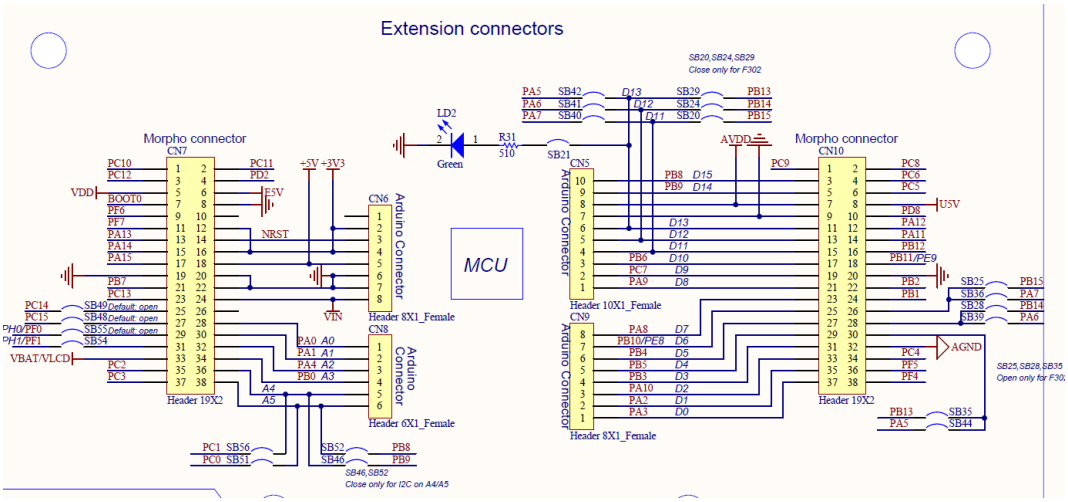
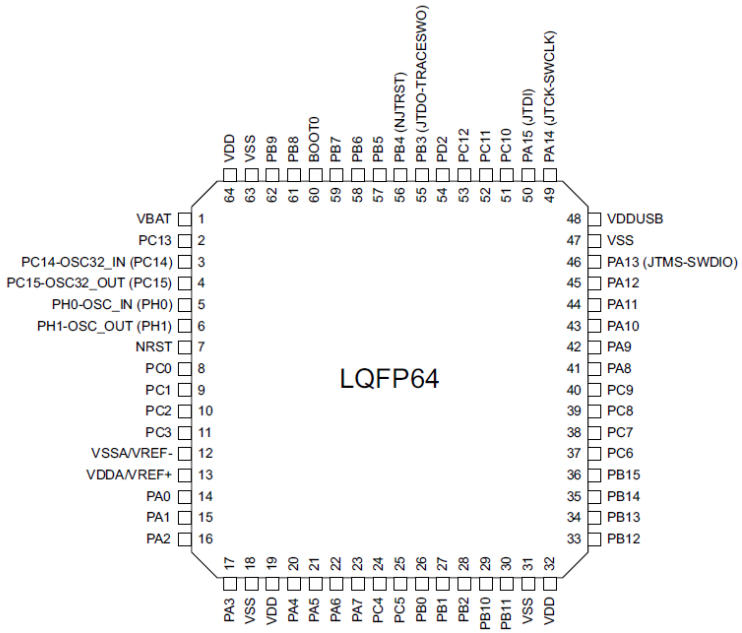


Figure : STM32L476RG Nucleo board Connectors

The datasheet also provides the view of the component package with the pins



A table with all possible features on a pin is also available with an example below

Pin Number											Pin name (function after reset)	Pin type	I/O structure	Notes	Pin functions	
LQFP64	LQFP64_SMPS	WLCSP72	WLCSP72_SMPS	WLCSP81	LQFP100	UFBGA132	UFBGA132_SMPS	LQFP144	LQFP144_SMPS	UFBGA144					Alternate functions	Additional functions
21	21	H6	H5	H6	30	K4	K4	41	41	M3	PA5	I/O	TT_a	-	TIM2_CH1, TIM2_ETR, TIM8_CH1N, SPI1_SCK, LPTIM2_ETR, EVENTOUT	ADC12_IN10, DAC1_OUT2
22	22	H5	J8	H5	31	L4	L4	42	42	K4	PA6	I/O	FT_la	-	TIM1_BKIN, TIM3_CH1, TIM8_BKIN, SPI1_MISO, USART3_CTS, QUADSPI_BK1_IO3, LCD_SEG3, TIM1_BKIN_COMP2, TIM8_BKIN_COMP2, TIM16_CH1, EVENTOUT	OPAMP2_VINP, ADC12_IN11

Further information on the card is available in the user manual UM1724.

4.2. Clock management

The STM32L476 is a 32-bit "low power" µcontroller, i.e. intended for applications that require little energy. To optimize the energy consumption of a µcontroller, several methods have been implemented, including clock management based on the principle that the faster the clock (High), the more current the µcontroller consumes.

- The first way is a clock management system that allows you to choose the clock source and the frequency at which the µcontroller will work. The figure below shows that the clock sources are:
    - internal HSI (High Speed Internal), MSI (Multi Speed Internal), LSI (Low Speed Internal)
    - external HSE (High Speed External) or LSE (Low Speed External) depending on whether an external quartz is used or not.

- The second way is to enable or disable the peripheral clock. Indeed, if the clock of a peripheral is disabled then it is inactive and consumes a minimum of energy. When all the peripherals remain "connected" to the clock, the µcontroller consumes much more than if we just connect the peripherals actually used.

So there is a "(RCC) Reset and Clock Control" device entirely dedicated to managing the clock to the devices.

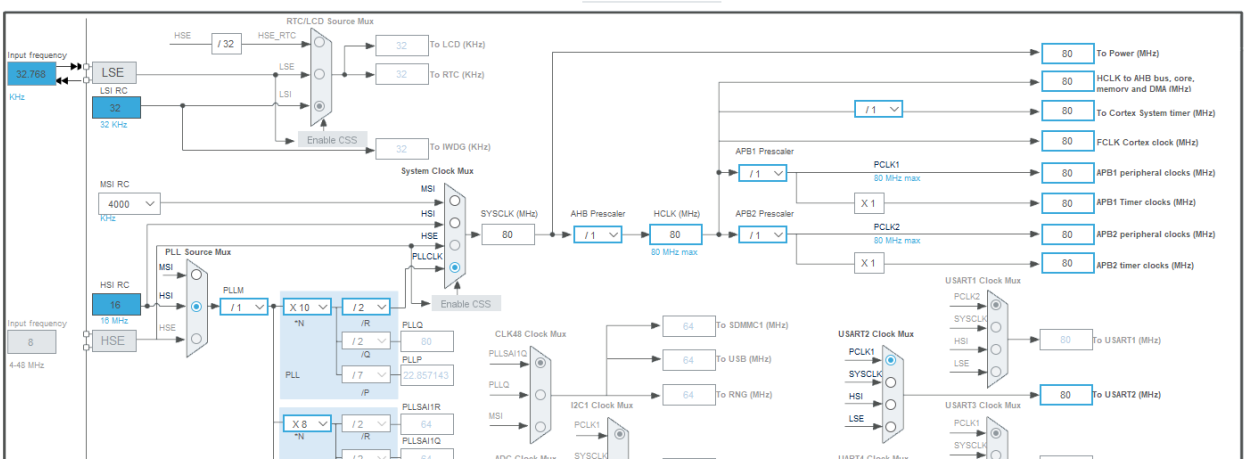
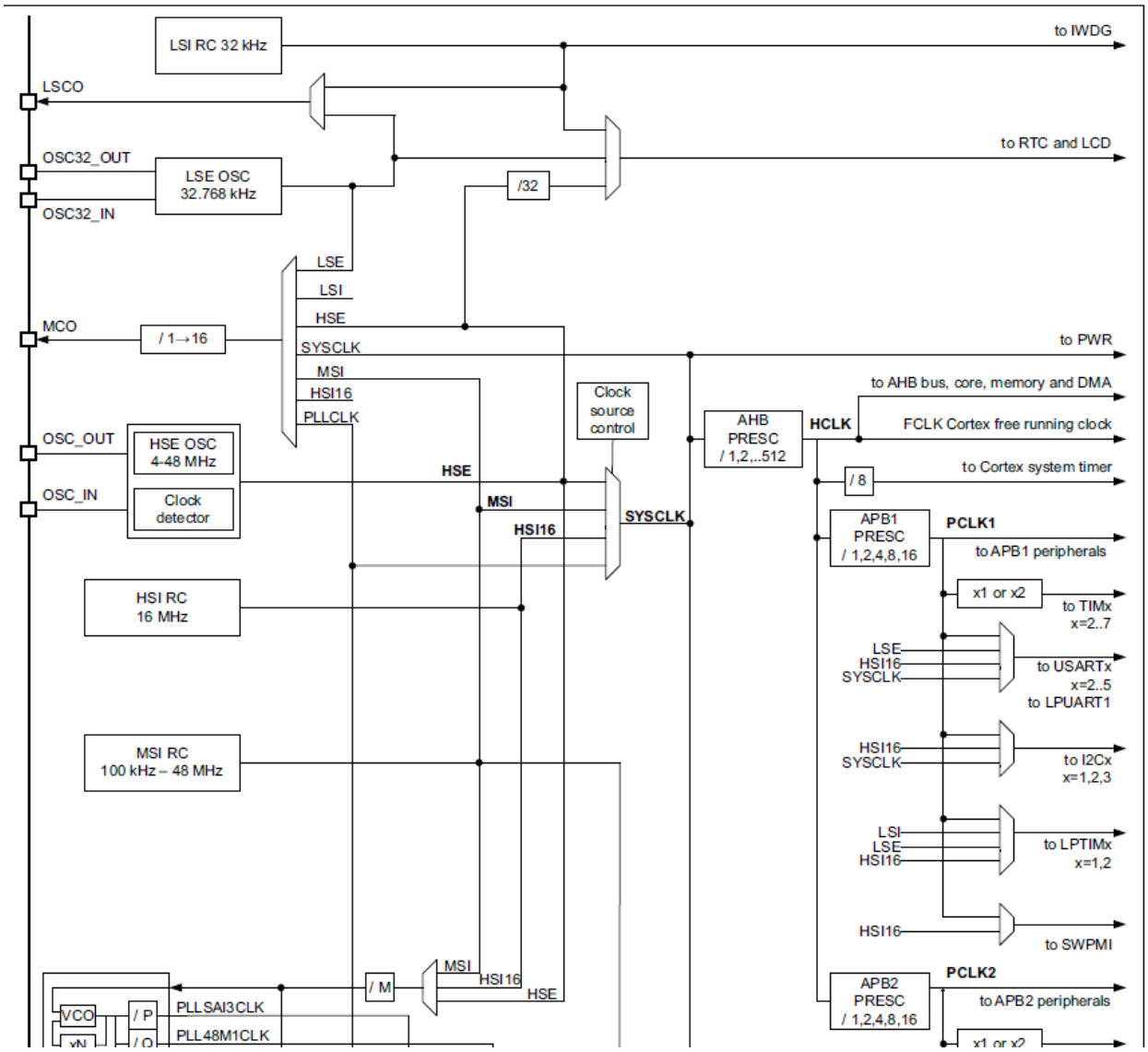


Figure 1 : clock tree block diagram

The configuration used during the labs will be the default configuration at reset: MSI ON The fCPU frequency is then 4 MHz. To limit consumption, all peripheral clocks have been disabled. **You will be asked to enable the clock at the peripheral level when necessary.**

The devices are connected via the internal buses AHB1, AHB2, APB1 or APB2, so you will need to power on and provide a clock to the devices before you can access them.



To do this you will need to read and understand the functions of the RCC device registers.

Chapter 5. Inputs / Outputs

A microcontroller has input/output ports (I/O or GPIO) to communicate with the outside world. Each I/O (GPIO) of an input/output port can be configured as an input to test the state of a logic signal or configured as an output to generate a logic signal. In this chapter, we will use the input/output ports of the STM32 microcontroller to control the switching on or off of LEDs.

5.1. Operating Principle

The pins are grouped into ports and an I/O port can have up to 16 I/Os. Each I/O can be configured independently by simply acting on the bit of the registers where it is assigned. The STM32 has 10 different registers for each port to interact with the I/Os. The name and function of the registers are given in the following table.

MODER: Mode Register (input; output, alternate function, analog)
OTYPER: Output Type Register (Output Speed Register)
PUPDR: Pullup / Pull-down register (if pin configured as open drain)
IDR: Input Data Register
ODR: Output Data Register
BSSR: Bit Set / Reset Register
LCKR: Lock Register
AFRL/H: Alternate Function Register (connect pin to timers, bus, event)
BRR: Bit Reset Register (reset ODR registers)

The GPIOx_ODR direction register controls the voltage level of each bit/pin n of port x.

GPIOx_ODR Status bit n	**0**	**1**
Port x bit n	Pin at 0v	Pin at 3,3V

Since a port can control up to 16 pins, the 32-bit ODR register has 16 bits to control the voltage level of the pins and 16 bits that are functionless (reserved)

8.5.6 GPIO port output data register (GPIOx_ODR) (x = A to I)

Address offset: 0x14

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
OD15	OD14	OD13	OD12	OD11	OD10	OD9	OD8	OD7	OD6	OD5	OD4	OD3	OD2	OD1	OD0
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bits 31:16 Reserved, must be kept at reset value.

Bits 15:0 **OD[15:0]:** Port output data I/O pin y (y = 15 to 0)
These bits can be read and written by software.

Note: For atomic bit set/reset, the OD bits can be individually set and/or reset by writing to the GPIOx_BSRR register (x = A..FA to H).

5.2. Initializing input/output ports

All programs that use the I/O ports must have a subroutine to initialize the registers controlling the controller ports. This initialization subroutine placed at the beginning of the program configures the I/O ports whose default value does not correspond to the needs.
You will therefore have to write a subroutine "init_ports" that initializes the ports in the requested mode.

When you use the registers associated with the I/O ports, you must keep the state of the bits that you do not modify, hence the need for masks.

5.3. Detailed characteristics

Details on the I/O operating modes are given in the STM32l152 Family Reference Manual RM0351 in the General Purpose I/Os section.

The following block diagram details the control and structure of an input/output:

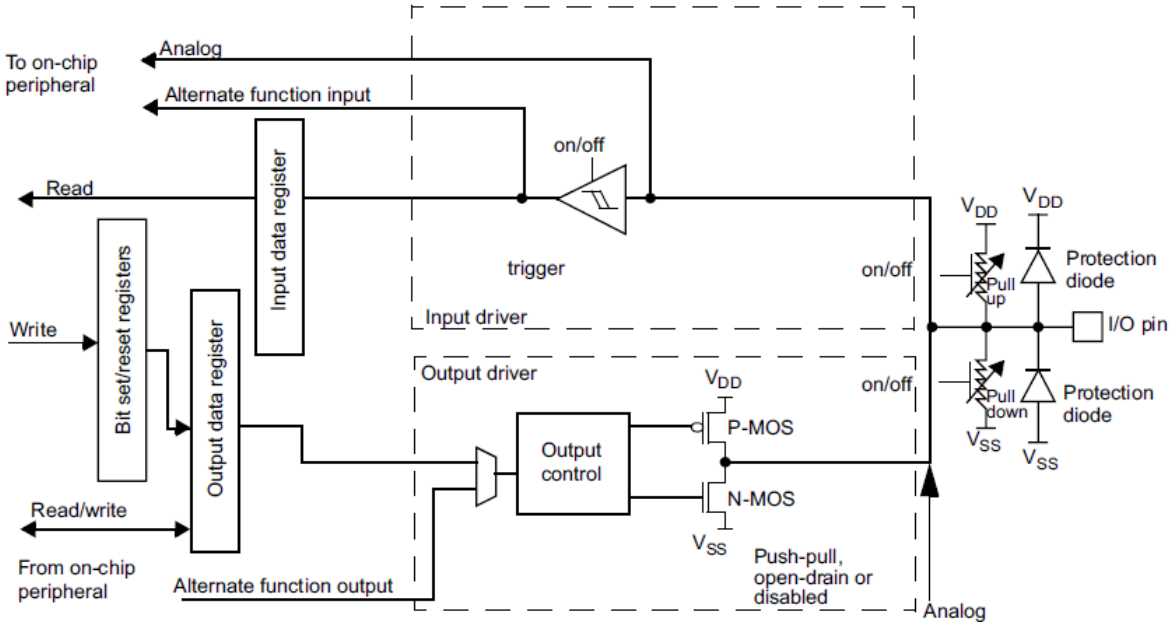


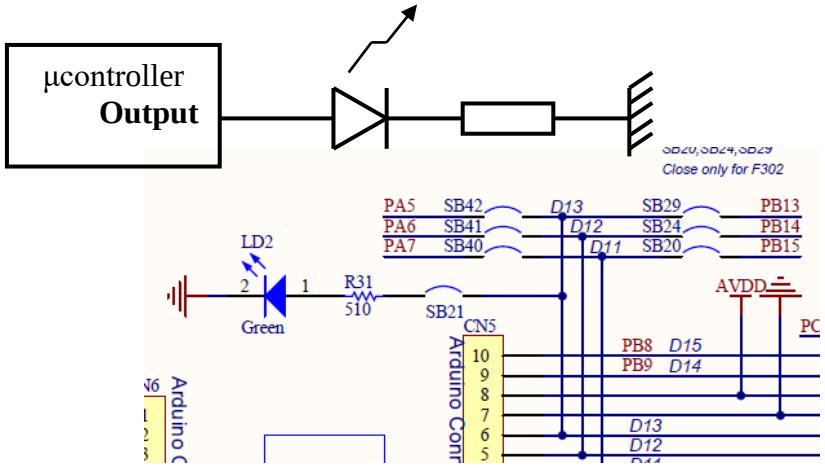
Figure 2 : Schéma bloc d'une E/S

The pin can be configured as input/output, push-pull or Open Drain, used as analog or digital, connected to another peripheral (timer, CAN, ...).
These numerous possibilities explain the high number of registers to configure a pin.

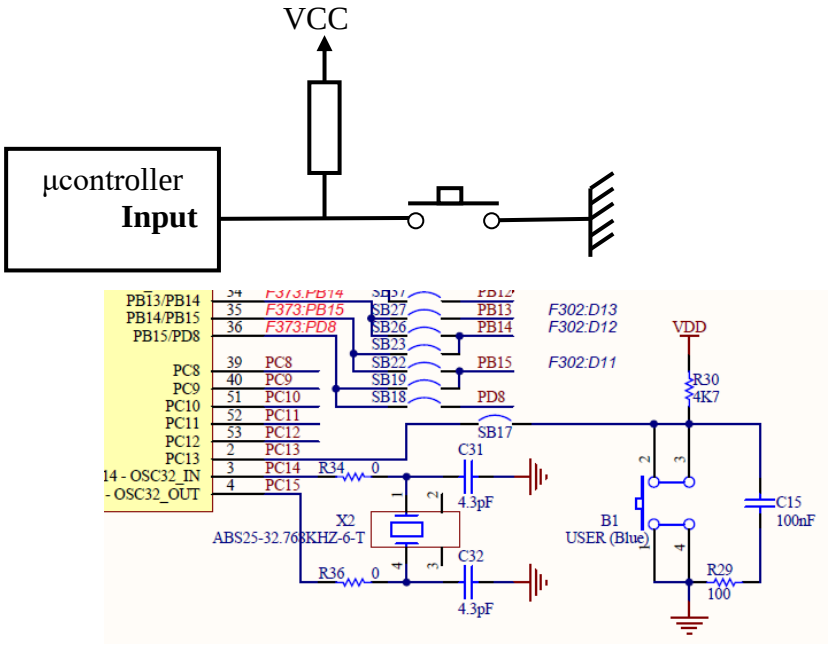
5.4. Controlling an LED with a push button

This exercise involves lighting up the LED when the push button is pressed.

The LED is controlled by bit 5 of port A in push-pull output as shown in the following figure:



The state of push button 1 is seen by bit 13 of port C (PC13) as input as shown in the following figure:



You have to :

- Write a subroutine "init_pin_LED" to initialize the pin that controls the LED. The LED is connected to PA
  - In the RCC device, turn on the AHB bus clock for the GPIOA device
  - Configure pin PA5 as output.

- Write a subroutine "init_ports_BP" to initialize the pin connected to the push button. The push button is connected on PC13

- In the RCC device, turn on the AHB bus clock for the GPIOC device
- Configure PC13 in input mode

- Write a subroutine "turn_on_LED" that lights the desired LED without changing the status of the other pins
  - Use the ODR or BSRR register of port A

- Write a subroutine "turn_off_LED" which turns off the desired LED.
  - Use the ODR or BSRR register of port A

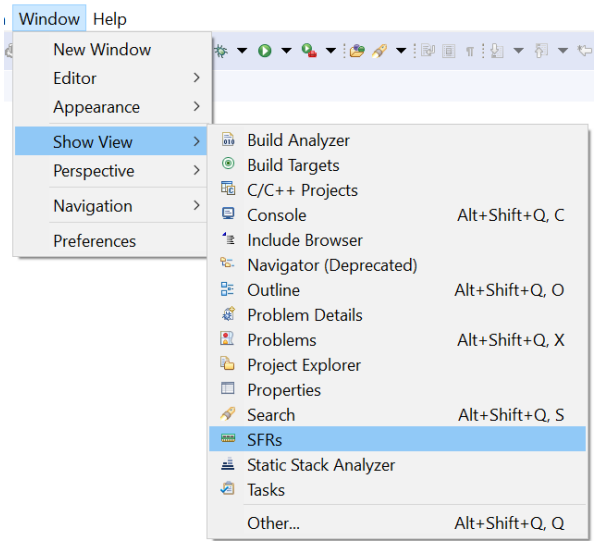
- Write a subroutine "status_push" which reads port C and indicates the state of the pusher in the variable "push".
  - Use the IDR register of port C

- Write your main program that calls "init_ports_LED" and "init_ports_BP" and then loops infinitely on:
  1. Call " status_push "
  2. Call "turn_on_LED" or "turn_off_LED" in conjunction with the variable "push"

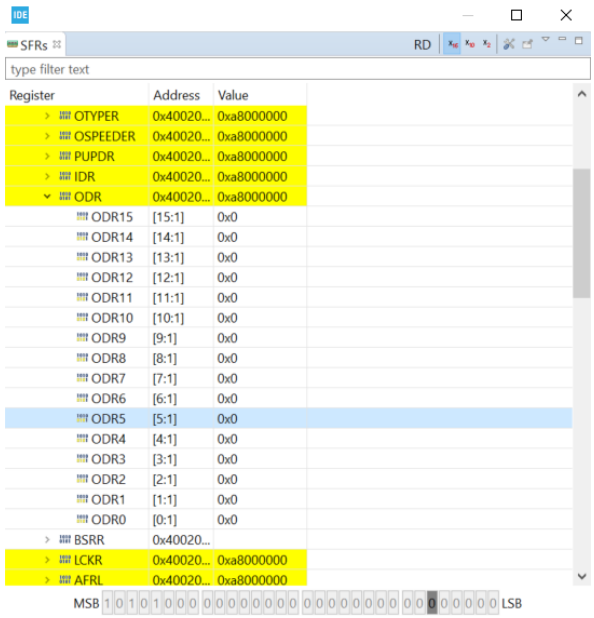
- Testez votre programme afin de vérifier son bon fonctionnement.

Note : If you use a function like bl<c> with an error message "Error: thumb conditional instruction should be in IT block", code the IT<c> function first. Ask your teacher for help if necessary.

Note: the configuration and status of a port can be viewed with the debugger in step-by-step mode by displaying the SFR window using the "Show View" menu.



It is also possible to modify the value of a register directly in this window by clicking directly on the bits



5.5. LED flashing

This exercise consists of turning on LED #1, waiting 0.5 seconds, then turning it off, waiting another 0.5 seconds, and starting again.
The wait between the different phases is done by a timing subroutine. A timing subroutine is a subroutine that performs "useless" operations for the desired time, either by loops (sometimes nested) or by other methods. The timing duration is found by calculating the number of cycles of the subroutine (the instruction documentation gives the number of cycles of each instruction).

You have to :

- Calculate the number of cycles of your 0.5 second timing subroutine given that the fCPU frequency is 4 MHz

- Write a subroutine "tempo_500ms" that performs the desired timing.

- Write your main program that must call "init_ports_LED", then successively "allume_LED", "tempo_500ms", "eteint_LED", "tempo_500ms" and loop on "allume_LED" to start again.

- Test your program

- **Check the duration of your timer by timing the duration of 30 flashes.**

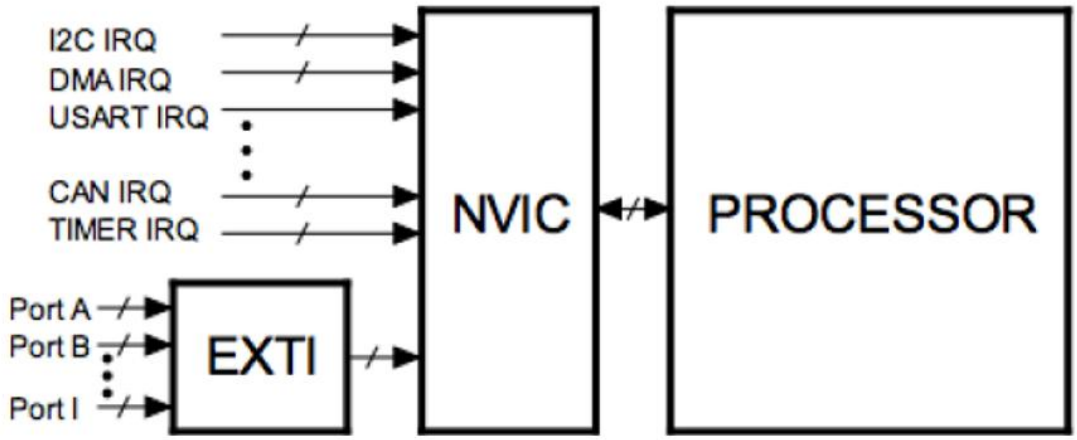
Chapter 6. Interrupts

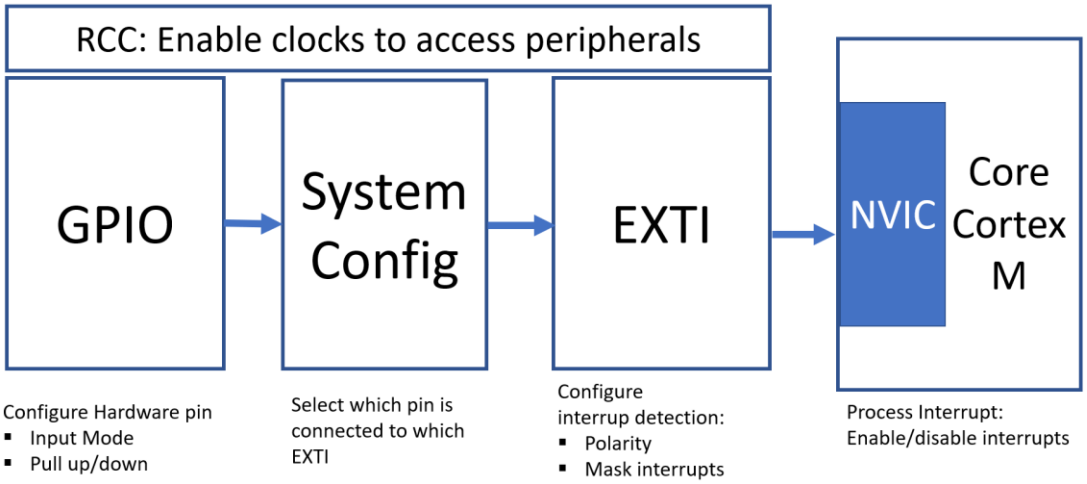
6.1. Principle of handling external interruptions

In order to configure an interrupt on a pin with the STM32, 5 blocks are involved

- ✓ RCC: turn on the clocks to be able to write to the peripherals
- ✓ GPIO to put the pin in input
- ✓ System Config to know which pin is configured to which "external interrupt"
- ✓ EXTI to manage the polarity: rising edge, falling edge or both
- ✓ NVIC (Nested Vectored Interrupt Controller) belongs to the Core of the Cortex M3 and manages the interrupts (on or off, priority ...).

You will therefore have to configure each block correctly. If a block is not configured properly, the interrupt does not work although the program compiles





Interrupt vectors are declared at the end of the startup file. The first interrupt (Reset) is at address 0x0000 0004 (see Figure 4). Each interrupt is identified by an address, a number and a name. The name is set by ARM to ensure program portability between different silicon vendors.

0	7	settable	WWDG	Window Watchdog interrupt	0x0000 0040
1	8	settable	PVD_PVM	PVD/PVM1/PVM2/PVM3/PVM4 through EXTI lines 16/35/36/37/38 interrupts	0x0000 0044
2	9	settable	RTC_TAMP_STAMP /CSS_LSE	RTC Tamper or TimeStamp /CSS on LSE through EXTI line 19 interrupts	0x0000 0048
3	10	settable	RTC_WKUP	RTC Wakeup timer through EXTI line 20 interrupt	0x0000 004C
4	11	settable	FLASH	Flash global interrupt	0x0000 0050
5	12	settable	RCC	RCC global interrupt	0x0000 0054
6	13	settable	EXTI0	EXTI Line0 interrupt	0x0000 0058
7	14	settable	EXTI1	EXTI Line1 interrupt	0x0000 005C
8	15	settable	EXTI2	EXTI Line2 interrupt	0x0000 0060
9	16	settable	EXTI3	EXTI Line3 interrupt	0x0000 0064
10	17	settable	EXTI4	EXTI Line4 interrupt	0x0000 0068
11	18	settable	DMA1_CH1	DMA1 channel 1 interrupt	0x0000 006C

Below the interrupt vectors, the functions that are called when an interrupt occurs are already defined. Replace Default_Handler with the name of the function you will call.

```
594
595     .weak   EXTI0_IRQHandler
596     .thumb_set EXTI0_IRQHandler,Default_Handler
597
598     .weak   EXTI1_IRQHandler
599     .thumb_set EXTI1_IRQHandler,Default_Handler
600
601     .weak   EXTI2_IRQHandler
602     .thumb_set EXTI2_IRQHandler,Default_Handler
603
604     .weak   EXTI3_IRQHandler
605     .thumb_set EXTI3_IRQHandler,Default_Handler
606
607     .weak   EXTI4_IRQHandler
608     .thumb_set EXTI4_IRQHandler,Default_Handler
609
```

6.2. Flashing with on/off with interrupt

This exercise consists of taking the "flashing an LED" exercise from chapter 5 and adding the management of pressing the blue push button connected to pin PC13 using interrupts
:

- Pressing the push button starts the LED flashing, starting with the LED on for 0.5s.
- Pressing the push button again stops the LED flashing (LED off after 0.5s maximum).

The state change will be when the button is pressed and not when the button is released. So you need to study the schematic and check how this button is designed to decide which polarity to configure.
You have to :

- Create a "flashing" variable which will indicate the state of the flashing light: on or off.
- Initialize the STM32 so that PC13 is configured for the interrupt and PA5 to manage the LED. You will create 2 subroutines init_led and init_interrupt_ext . For a pin to be configured as an interrupt, the following registers must be programmed:
    - Enable GPIOx device clock
    - Enable clock of the block system configuration which is connected to the APB2 bus
    - Configure interrupt polarity
    - Enable interrupt in interrupt controller block
    - Configure the NVIC to enable interrupt

- Create in the declaration area of the interrupt subroutines, "on_off_interrupt" which will be called each time PC13 is pressed.

- Test the program to verify that it is working properly.

- Note the evolution of the PC, SP, LR and xPSR registers at the entry and exit of the interrupt

- Identify the information saved in the stack.

Interrupt Input and Output:

Check the value of registers R0, R1, R2, R3, SP, PC, LR and xPSR, before, during and at the end of an interrupt. What happens automatically at each interrupt?

Program Counter PC	Stack Pointer SP	Stack											Reg LR	R0	R1	R2	R3	xPSR	NOTES

Appendix 1. Conditional jump instructions

cond	Mnemonic extension	Meaning (integer)	Meaning (floating-point) [ab]	Condition flags
0000	EQ	Equal	Equal	Z == 1
0001	NE	Not equal	Not equal, or unordered	Z == 0
0010	CS [c]	Carry set	Greater than, equal, or unordered	C == 1
0011	CC [d]	Carry clear	Less than	C == 0
0100	MI	Minus, negative	Less than	N == 1
0101	PL	Plus, positive or zero	Greater than, equal, or unordered	N == 0
0110	VS	Overflow	Unordered	V == 1
0111	VC	No overflow	Not unordered	V == 0
1000	HI	Unsigned higher	Greater than, or unordered	C == 1 and Z == 0
1001	LS	Unsigned lower or same	Less than or equal	C == 0 or Z == 1
1010	GE	Signed greater than or equal	Greater than or equal	N == V
1011	LT	Signed less than	Less than, or unordered	N != V
1100	GT	Signed greater than	Greater than	Z == 0 and N == V
1101	LE	Signed less than or equal	Less than, equal, or unordered	Z == 1 or N != V
1110	None (AL) [e]	Always (unconditional)	Always (unconditional)	Any

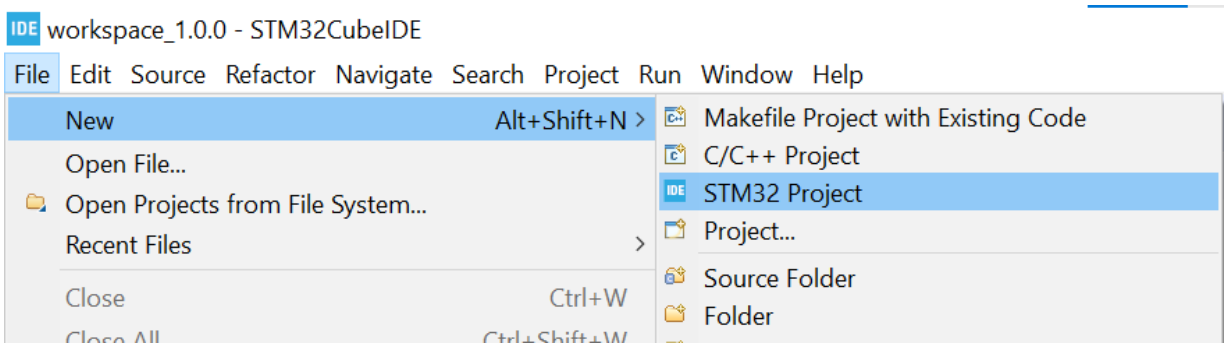
Appendx 2. Create an STM32 project

The STMicroelectronics development kit dedicated to STM32 family microcontrollers allows you to easily create projects with a graphical environment.

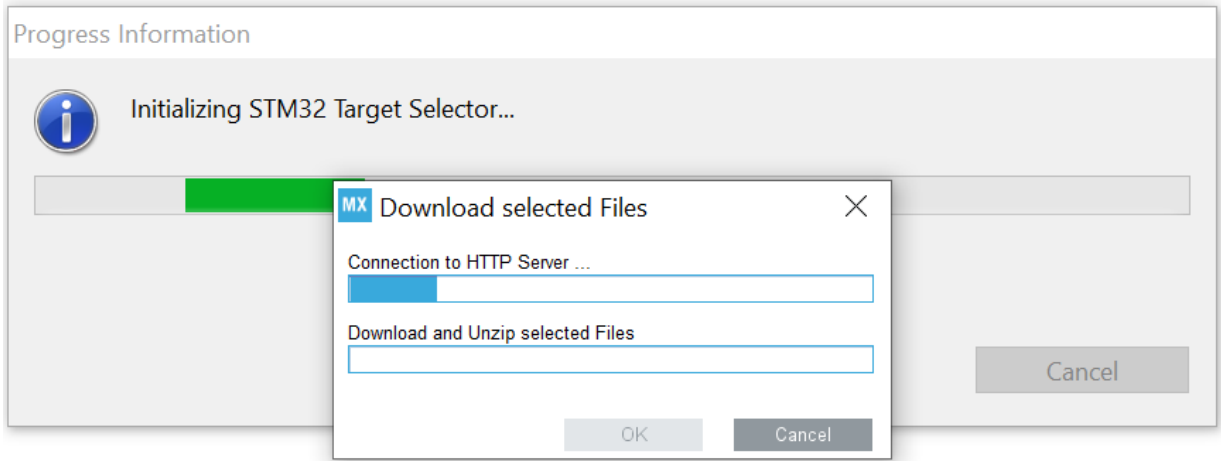
This booklet will guide you step by step in the creation of such a project.

Each step will be detailed and illustrated with screenshots to facilitate your reading.

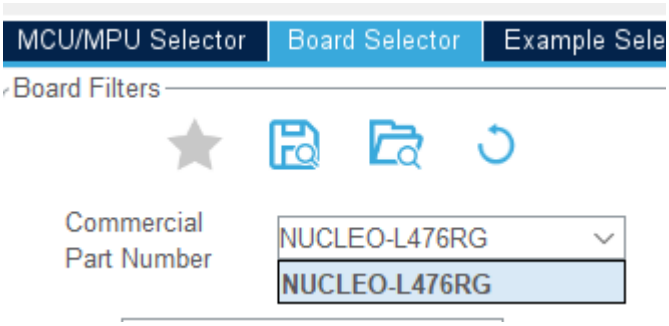
After reading this short guide, you will be able to create your first project and test it
Pour créer un nouveau projet, cliquer sur File/New/STM32 Project



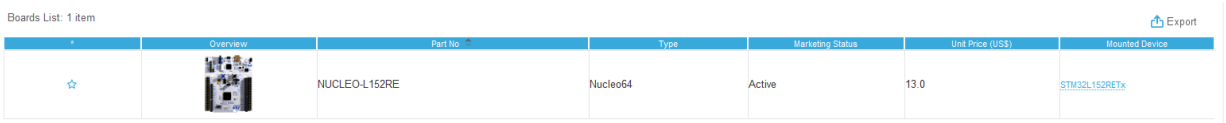
If an internet connection is available, STM32CubeIDE can update the available product offering. This update may take a few seconds.



Select the Board Selector tab and in the Parr Number Research window select Nucleo-L476RG



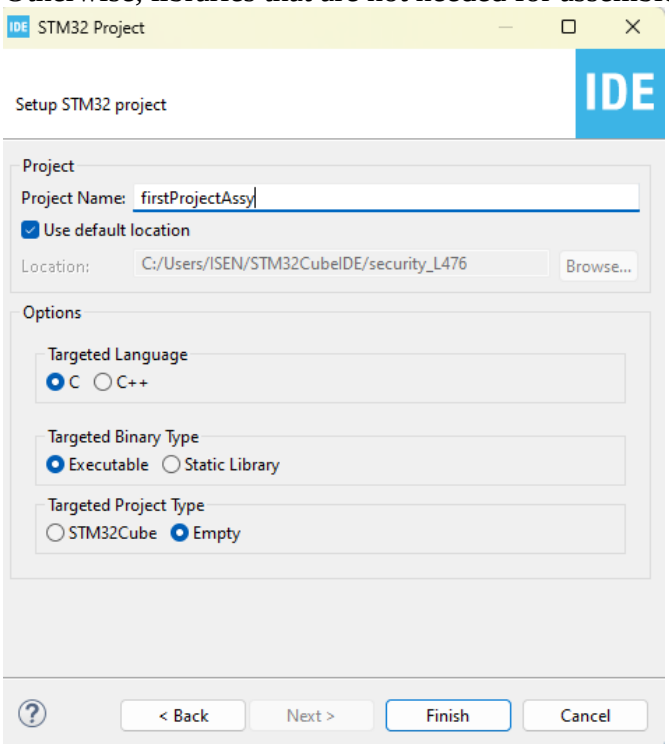
On the right of the window, the map used for the practical work appears.

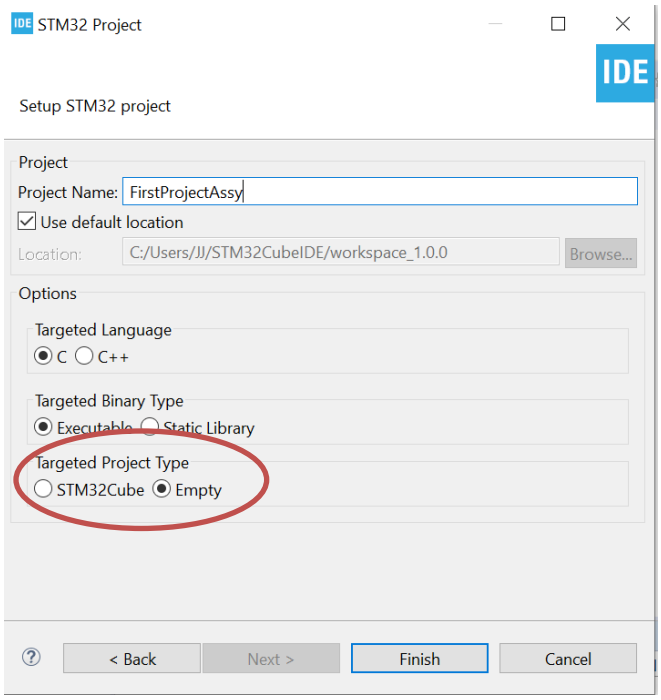


Enter the project name.

!!! In Targeted Project Type, select Empty !!!

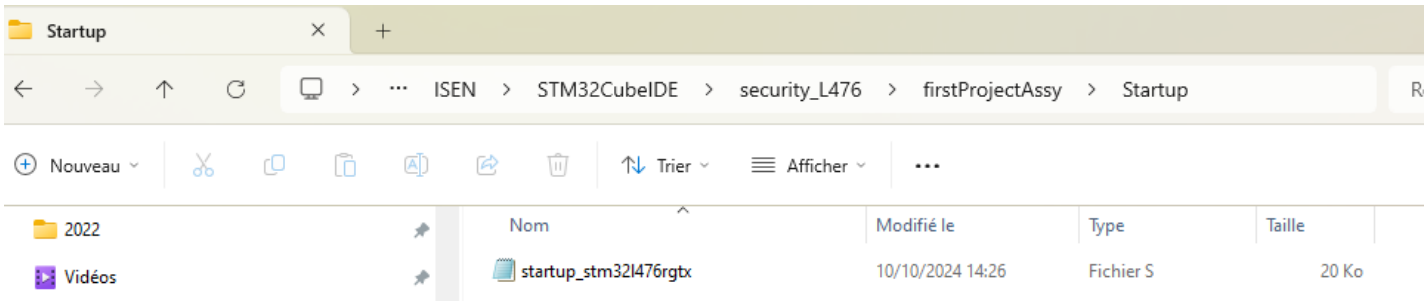
Otherwise, libraries that are not needed for assembler labs will be added to the project.





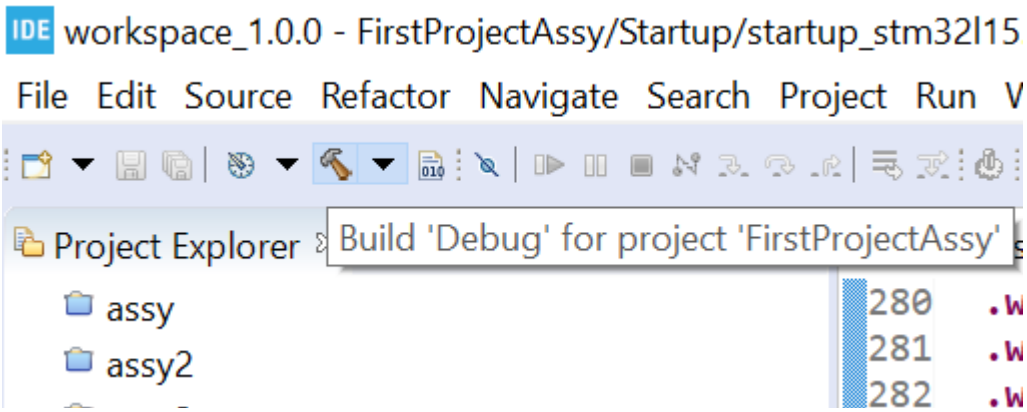
Click on Finish.

With Windows Explorer in the workspace you created and in the start up folder.



Warning: Replace the startup_stm32l476rgtx.s file with the file of the same name published on the ENT.

Compile the program to check everything is set up correctly.




```
arm-none-eabi-size   FirstProjectAssy.elf
arm-none-eabi-objdump -h -S  FirstProjectAssy.elf  > "FirstProjectAssy.list"
   text    data     bss     dec     hex filename
    536       8    1568    2112     840 FirstProjectAssy.elf
arm-none-eabi-objcopy  -O binary  FirstProjectAssy.elf  "FirstProjectAssy.bin"
Finished building: default.size.stdout

Finished building: FirstProjectAssy.list

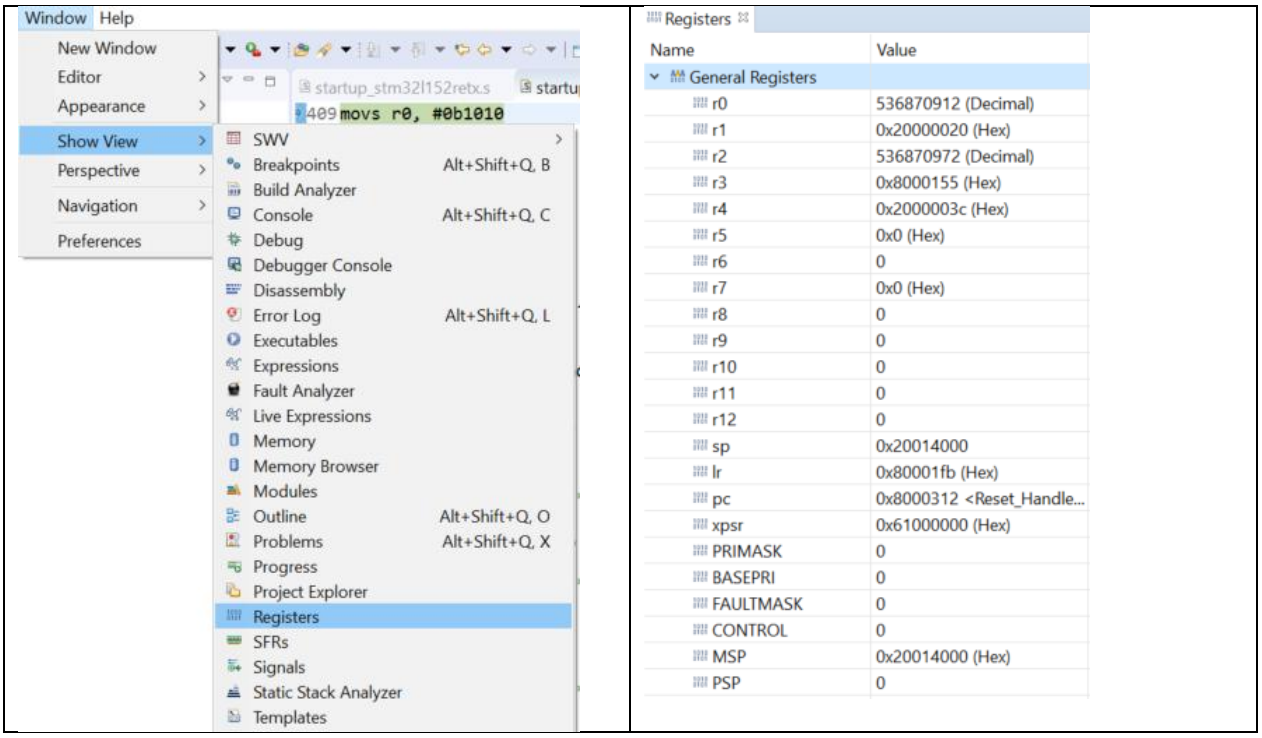
Finished building: FirstProjectAssy.bin


10:29:05 Build Finished. 0 errors, 0 warnings. (took 1s.491ms)
```

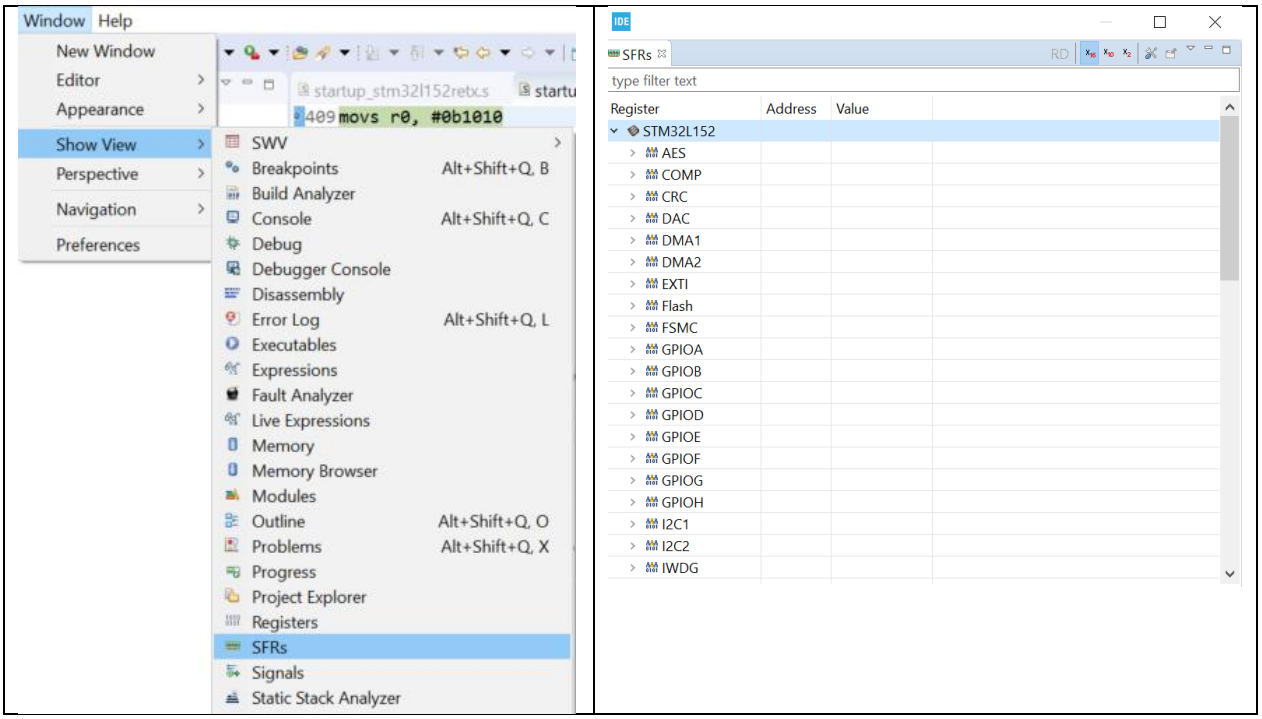
You are now ready to write your programs in assembler. The windows presented in Appendix 3 will be necessary to answer the practical questions and debug your program.

Appendix 3. Debugging Windows

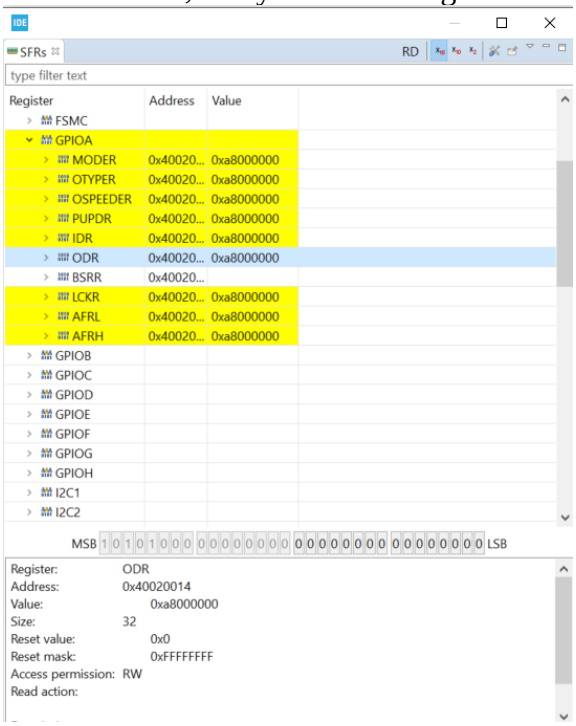
Window to access internal registers



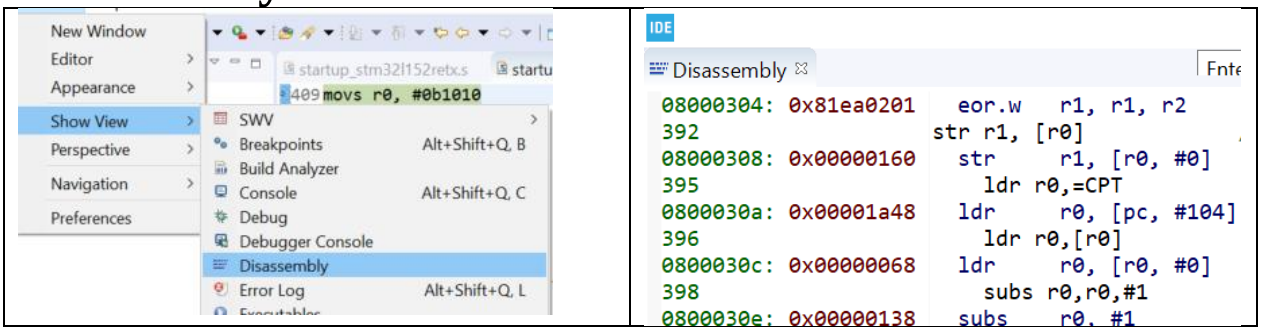
Window to access peripherals



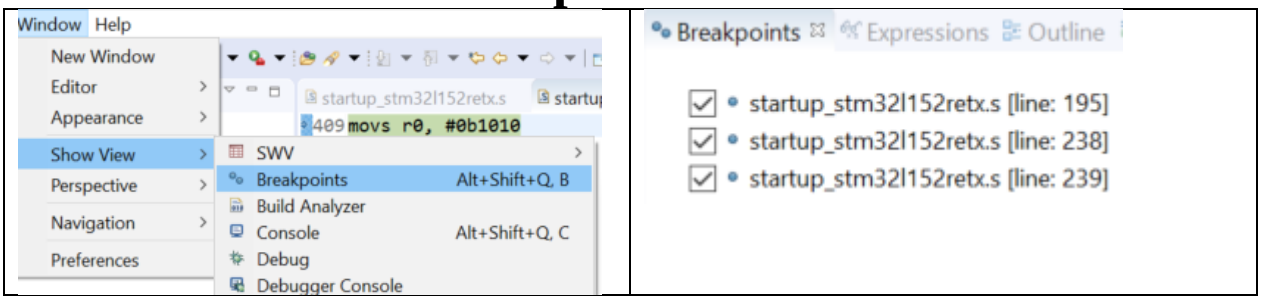
And of course, every detail of a register is accessible



Disassembly window



Window to access breakpoints



To create a breakpoint, double-click in front of the line. To cancel a breakpoint, in the breakpoint window, click on a breakpoint and press the ‘del’ key.

Window to access a memory address

